任务四技术报告

一、技术要求

二、具体技术实现

1.A*算法简介

2.A*优化思路

2.1从算法出发

2.11双向A*（bidirectional Astar search)

2.12模拟退火

2.13针对障碍物的优化

2.14Theta*和Lazy Theta*

2.15二次搜索

2.2从启发函数出发

2.21getHeu

2.22getAngle

2.23添加权重w

2.24对于相等f值选择的处理

2.3从数据结构出发

2.31A*

2.32在点云数据中建立空间索引

3.拓展部分（用python实现）

3.1基于采样的RRT系列算法

3.12RRT

3.13RRT*

3.14informed RRT*

3.15ompl库

3.2局部路径优化

3.21DWA

3.22teb

3.3基于数学的路径优化

哈工大威海创新平台 - 718-夏天怡(22970116)

3.31多项式轨迹
3.32MinimumSnap
3.4PRM(概率路线图)
3.5常见路径规划算法的总结：
3.6Navigation Mesh
4.一些调试和比较的小技巧
4.1地图固定
4.2同时显示路径
4.3调整地图大小和起点
4.4 visited_nodes
4.5rviz显示visited nodes
4.6利用ROS_WARN来调试
5.错误修正
5.1双向A*
5.2编译的一些问题
5.3模拟退火
5.4getAngle
三、算法部署
四、效果展示
五、未来优化与应用
六、总结
七、参考资料

一、技术要求

1. 搭建并配置Linux系统与ROS环境
2. 查找并学习A*路径规划算法
3. 对A*算法进行优化

二、具体技术实现

1.A*算法简介

哈工大威海创新平台 - 718-夏天怡(22970116)

A*算法通过下面这个函数来计算每个节点的优先级。

$$f(n) = g(n) + h(n)$$

其中：

- f(n)是节点n的综合优先级。当我们选择下一个要遍历的节点时，我们总会选取综合优先级最高（值最小）的节点。
- g(n) 是节点n距离起点的代价。
- h(n)是节点n距离终点的预计代价，这也就是A*算法的启发函数。

A*算法在运算过程中，每次从优先队列中选取f(n)值最小（优先级最高）的节点作为下一个待遍历的节点。另外，A*算法使用两个集合来表示待遍历的节点，与已经遍历过的节点，这通常称之为open_set和close_set。

A*的完整实现思路如下：

- 初始化 `open_set` 和 `close_set`
- 将起点加入 `open_set` 中，并设置优先级为0（最高）

这里的优先级其实就是A*算法中最重要的启发函数

- 如果 `open_set` 不为空，则从 `open_set` 中选取优先级最高的节点n：
  - 如果节点n为终点，则：
    - 从终点开始逐步追踪parent节点，一直达到起点
    - 返回找到的结果路径，算法结束；
  - 如果节点n不是终点，则：
    - 将节点n从 `open_set` 中删除，并加入 `close_set` 中
    - 遍历节点n所有的邻近节点
      - 如果邻近节点m在 `close_set` 中，则：
        - 跳过，选取下一个邻近节点
      - 如果邻近节点m没有被访问过，则：
        - 设置节点m的parent为节点n
        - 计算节点m的优先级
        - 将节点m加入 `open_set` 中
      - 如果邻近节点m在 `open_set` 中，则：
        - 计算父节点是否需要被更新
- 其中，启发函数会影响A*算法的行为。

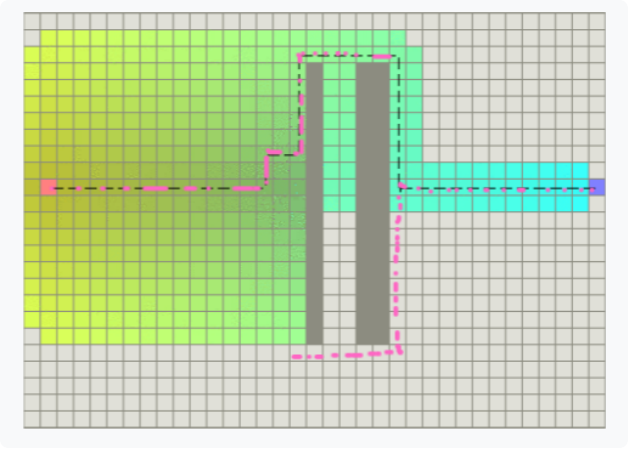
2.A*优化思路

2.1从算法出发

2.11双向A*（bidirectional Astar search）

在对传统A*算法进行实现和分析后，我们发现它实际运行时间性能较差，并且存在一些缺点，很难满足实际运用的需求，而基于A*算法，很多改进的算法也应运而生，双向A*就是其中之一。与传统A*不同的是，双向A*算法采取的搜索方式是从起点和终点进行两个方向的搜索，一个向终点扩展，另一个向起点扩展，从理论上说，两个搜索路径汇合到同一个位置时，就可以得到一条最短路径。

但是在实际使用过程中我们发现，由于两个方向的搜索之间没有关联性，它们很容易产生“互不认识”的情况，如下图。



在某些场合的情况下存在较难的汇合情况，或者“搜索路径汇合到同一个位置”的要求其实在很多情况下难以实现，对此，我们改变搜索停止的要求，了解到有以下两种方式改进：

1. 将“汇合到同一个位置”的限制放宽到两个节点的xyz坐标在一定范围之内即停止搜索，通常这个范围是略大于障碍物尺寸的，然后记录当前两节点的位置，于是就将最终的路径分为了三段。
2. 采用“交替”搜索的方式，在AstarGraphSearch内while里先进行一次正向搜索再进行一次反向搜索，re-表示反向

考虑到方法1还需要在两个方向的终止节点之间再进行一次Astar搜索，代码的结构不够清晰，我使用了方法2，它的具体实现思路如下：

（1）Astar从起点出发，reAstar从终点出发

（2）Astar以reopen表中的F值最小的栅格为终点进行扩展；

（3）reAstar以open表中的F值最小的栅格为终点进行扩展；

（4）若open表or reopen表为空，表示找不到，退出；

（5）重复（2）（3）（4），直到满足设置好的终止条件，此时两者相遇，表示成功找到路径，退出；

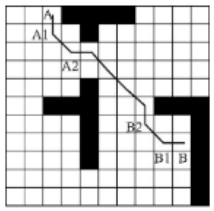


图3 双向A*算法的搜搜示意图

（6）而终止条件，我给出的是：如果在正向查找表fopenSet中找到了此时正向搜索的目标点(recrtPtr)，或者在反向查找表freopenSet中找到了此时反向搜索的目标点(crtPtr)，查找终止。以正向搜索时停止为例，将recrtPtr在fopenSet里对应的节点p找到以后（这两个节点实际上是同一个节点但是它们的camefrom不一样），返回该节点p和recrtPtr给getPath，由p回溯父节点，得到正向路径，由recrtPtr回溯父节点，得到反向路径。
由于官方给的例程中openSet使用数据结构multimap二元映射存储的，只能根据key值(fScore)对节点指针进行遍历，高于了我对查找节点的需要，所以我重开两个数据结构list<GridNodePtr>fopenSet和freopenSet，起到类似于openset但是只用于查找节点，且以节点信息存储的功能。然后查找功能的实现就是利用迭代器从begin到end–1(*后改成end–1到begin*)遍历一遍进行查找即可。
使用的数据结构：

```
std::multimap<double, GridNodePtr> openSet;
std::multimap<double, GridNodePtr> reopenSet;
```

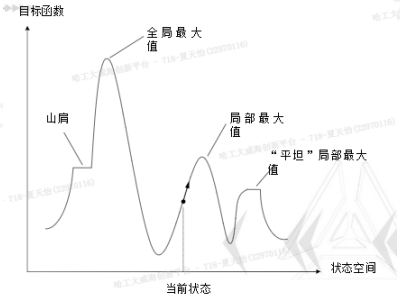
```
std::list<GridNodePtr> fopenSet;
std::list<GridNodePtr> freopenSet;
```
两个用来查找的列表

2.12模拟退火

在实际的寻路过程中，很容易在一次搜索里陷入局部最优[1]，因此我们应该给予搜索打破局部最优僵局的可能性，方法有很多种，我参考了比较常见的模拟退火方式。
[1]:传统的 A*算法在从 openSet选取最小的 $f(n)$ 的节点作为currentPtr。从起始节点到目标节点搜索最短路径时，g(n)的数值是从父节点累加过来的，因此它可以尽可能地保证前面走过的路径是最短的。而h(n)代表从n节点到目标节点的启发函数的预测值，如果可以尽可能地保证 h(n)是逐渐减小的，那么在一定程度上就是保证 n节点后面的路径是在接近目的地。为了实现 n 节点后面的路径是在接近目的地，如果每次都以贪婪策略，即h（n）最小值节点来作为扩展节点是最合适的，但是这样就忽略了g(n)的影响，该算法也就变成了传统的深度优先搜索了，很有可能优化出来的路径只是局部最优。

因此,如果我们可以在扩展节点时对搜索做出一点干涉,来引导搜索朝着终点的方向进行,就可以很好的减少搜索时间和扩展节点数.

模拟退火是一种解决无约束和有边界约束的优化问题的方法。该方法模拟了加热材料然后缓慢降低温度以减少缺陷的物理过程，从而最大限度地降低了系统能耗。模拟退火算法包含两个部分，即Metropolis算法和退火过程。



模拟退火在路径规划中的使用方式如下：

1. 由上述分析,我们选择 $h(n)$ 作为模拟退火中的能量函数
2. 模拟退火的基本参数:
    a. 初温T=2,
    b. 退火速率 α = 0.98
    c. 初始 $h(n)$ = `getHeu(startPtr,endPtr)`
3. 待处理的节点是每一个由最小化 $f(n)$ 选出的currentPtr(n')
4. 计算 $\delta T = \ h(n') - h(n)$ 其中hn为上一个节点的启发函数计算值
5. 根据退火公式 $e^{-\delta T/T}$ 和产生的[0,1]之间的随机数p,如果 $e^{-\delta T/T} < p$ 则接受该节点n';若否,则在openSet里寻找 $h(n)$ 最小的节点作为n'.
6. 然后更新T, $T' = \alpha T$

代码应做如下修改：

哈工大威海创新平台 - 718-夏天怡(22970116)

模拟退火 C++ 复制代码

```cpp
//为新添加语句
//～～～为省略原代码
#include <cstdlib>
#include <ctime>

void AstarPathFinder::AstarGraphSearch(Vector3d start_pt, Vector3d end_pt)
{
    ros::Time time_1 = ros::Time::now();

    ～～～some basic initializations
    //double T = 2.0;
    //double a = 0.98;
    //double dT = 0.0;
    //double E = getHeu(startPtr,endPtr);
    //double p = 0.0;
    //srand(time(NULL));

    while ( !openSet.empty() ){
        T = a*T;
        p = rand()/double(RAND_MAX);
        int x = openSet.begin()->second->index(0);
        int y = openSet.begin()->second->index(1);
        int z = openSet.begin()->second->index(2);

        currentPtr = GridNodeMap[x][y][z];

        //dT = getHeu(currentPtr,endPtr) - E;
        /*
        E = getHeu(currentPtr,endPtr);
        if(dT<0)
            openSet.erase(openSet.begin());
        else if(exp(-dT/T)<p)
            openSet.erase(openSet.begin());
        else
        {
            //choose the node with smallest hscore
            auto i1 = openSet.begin();
            for(auto i = openSet.begin();i!= openSet.end();i++)
            {
                if((*i)->fScore - (*i)->gScore < (*i1)->fScore - (*i1)->gScore)
                    i1 = i;
            }
            x = i1->second->index(0);
```

```
            y = i1->second->index(1);
            z = i1->second->index(2);
            currentPtr = GridNodeMap[x][y][z];
        }
        */
        if(currentPtr->id == -1)
            continue;

        currentPtr->id = -1;

        ~~~


}

```

不过，由于此次线上赛的地图栅格不太多，路径也比较直观，openset里可选择的节点不太多，模拟退火的方法也没有必要使用，我也没有在最后的代码中使用.

2.13针对障碍物的优化

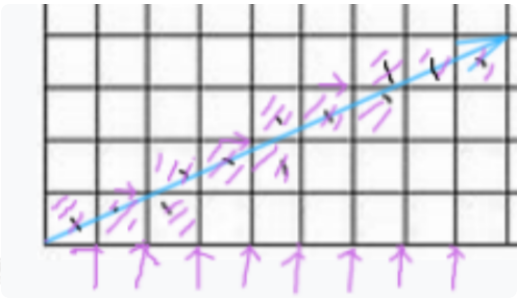
考虑到在当前位置和目标点连线之间，若无障碍物存在，可以直接退出A*算法沿连线直达目标点的情况，我引入 $sightdetect$ 来检测两节点间是否有障碍物.
之前使用这样一个方式去做sightdetect：（这个是正确的并且可以利用的）

哈工大威海创新平台 - 718-夏天怡(22970116)

伪代码 C++ 复制代码

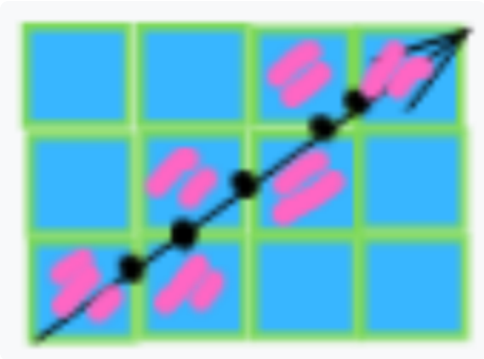
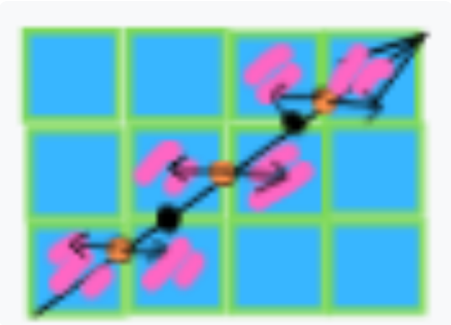
```
sightdetect(s,s1)
    x0 = s.x;y0 = s.y;
    x1 = s1.x;y1 = s1.y;
    dy = y1-y0;dx = x1-x0;
    f = 0;
    if(dy < 0)
    {
        dy = -dy;
        sy = -1;
    }
    else sy = 1;
    if(dx < 0)
    {
        dx = -dx;
        sx = -1;
    }
    else sx = 1;
    if(dx >= dy)
    {
        while(x0 != x1)
        {
            f+=dy;
            if(f>=dx)
            {
                if(grid(x0+((sx-1)/2),y0+((sy-1)/2))
                    return false;
                y0+=sy;
                f-=dx;
            }
            if(f!=0 && grid(x0+((sx-1)/2),y0+((sy-1)/2))
               return false;
            if(dy == 0 && grid(x0+((sx-1)/2),y0) && grid(x0+((sx-
1)/2),y0-1)
               return false;
            x0 += sx;
        }
    }
    else
    {
        while(y0 != y1)
        {
            f+=dx;
            if(f>=dy)
            {
                if(grid(x0+((sx-1)/2),y0+((sy-1)/2))
```

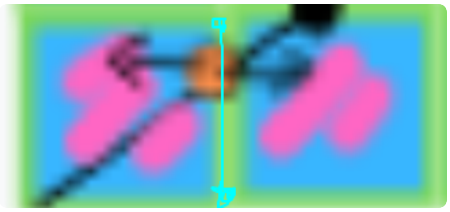
```
45                      return false;
46                  x0+=sx;
47                  f-=dy;
48              }
49              if(f!=0 && grid(x0+((sx-1)/2),y0+((sy-1)/2))
50                 return false;
51              if(dz == 0 && grid(x0,y0+((sy-1)/2)) && grid(x0-1,y0+((sy-
    1)/2))
52                 return false;
53              y0 += sy;
54          }
55      return true;
56  }
57
```

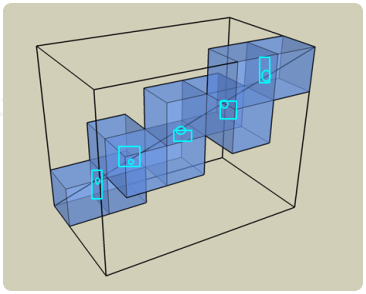


上述代码实现图示：

不过在我们线上赛的仿真环境中做这样的检测确实不容易：以下是我一点点尝试（虽然失败了）

> 但是在三维空间中去确定两个小立方体所构成的大立方体的体对角线穿过的所有正方体实际上是有一定难度的，于是我还是自己先从二维问题入手：
>
> 
> 
>
> 左下角坐标（0，0），右上角坐标（4，3）
> 对角线的直线方程为 $\frac{x}{4}=\frac{y}{3}$
> 那么我们可以取 $x \in (x1, x2)$ x为整数

得到多组对角线上的点（a,b）（用橙色点标出）（a为整数），然后利用两个for循环和if判断，将该点确定在一个范围内  ,判断左右方格是否是障碍即可。

推广至三维情况： 

在3d的栅格图里，设左下角立方体的左下角坐标为(x1,y1,z1)右上角立方体的右上角坐标为(x2,y2,z2)

对角线的直线方程为 $\frac{x-x1}{x2-x1}=\frac{y-y1}{y2-y1}=\frac{z-z1}{z2-z1}$

那么我们可以取 $a\in(x1,x2)$ a为整数

带入对角线方程就可以得到多组对角线上的点(a,b,c)

好像还是比较难，。。。

但是通过与学长的交流，在实际情况中，判断两点之间是否存在障碍物其实是比较容易的。比如说，可以利用激光雷达直接判断，也可以在仿真中利用建好的地图和定位解决问题。

不过，幸运的是，在我寻找line of sight检查的方法的时候，发现了这样一个算法“Navigation mesh”，为什么我要在这里提它呢，因为这里面用到了A*。具体见3.7。

2.14Theta*和Lazy Theta*

图2是Theta*方法， 可以看到在仅在计算ComputeCost(s,s')中和A*算法不同。在ComputeCost中，A*算法计算一个点s'的新的g，如果比原来的更好，则将原来的parent和g换成新的。在Theta*中，则在计算前，先去计算s'和s的parent有无**视线**。如果有视线，则把s'的parent和g以s的parent为parent进行更新。否则跟A*一样处理。这里视线的检查其实就是2.13中我对障碍物的优化 $sightdetect$ ，也就是说视线检查的结果可以引导两种操作：

1.跳出A*直接搜索，

2.Theta*，但是需它们的弊端要做大量的line of sight检查。对于2.有多少个点进入过open列表，就有多少次检查。在较为细致的网格中这个数量是十分巨大的。

所以就Lazy Theta*应运而生。（如图3）

在Theta*中，检查视线的时机发生于一个点进入open列表的时候。但是实际上，有很多进入open列表的点最终不在路径中，这意味着LOS检查是无效的。因此Lazy Theta*选择把LOS检查放到打开该节点的时候进行。

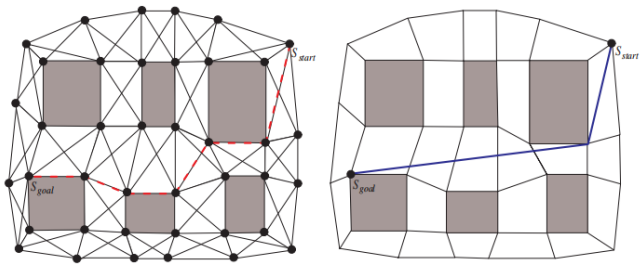
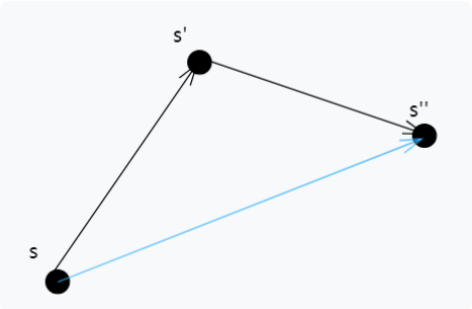


Figure 1: NavMesh: Shortest Path Formed by Graph Edges (left) vs. Truly Shortest Path (right), adapted from (Patel 2000)

由于在我们的仿真里体现不出anyangle的特征,所以这个算法没有被我实际使用.但是从中我得到了一个类似于这两个优化的新思路(2.15)。

2.15二次搜索



如图,对于得到的最终路径我们可以对其中的某个s''利用 `sightdetect` 判断它与s之间是否存在障碍物,如果没有,那么可以直接删除s与s''之间的节点,并且连接s与s'',将s作为s''的父节点。

C++ | 复制代码

```
vector<Vector3d> AstarPathFinder1::getPath()
{
    vector<Vector3d> path;//aggregate
    path.clear();

    ~~~
    int x = 40;
    for(auto itr = path.begin();itr!=path.end();itr++)
    {
        int num = rand() % 100 + 1;
        if(p<x)
        //有0.4的概率对改节点进行二次搜索，这样可以缩短二次搜索的时间
        {
            if(sightdetect(*itr,(*itr->camefrom)->camefrom))
            {
                path.erase(--itr);
            }
        }
    }
    return path;
}
```

2.2从启发函数出发

启发函数h(n)对于A*意味着什么：

启发函数值得大小会影响A*的具体实现，如果为0，那么自然演变成Dijkstra，如果较于g(n)非常高，那么A*又变成了最佳优先搜索；除此之外，由于我们不能准确地估计当前节点与终点之间的实际距离，所以我们在实际移动成本的附近取值，所以我们可以决定我们想从 A* 中得到什么。

有了 100% 准确的估计，我们将很快得到最短路径。如果我们太低，那么我们将继续获得最短路径，但它会变慢。如果我们太高，那么我们会放弃最短路径，但 A* 会跑得更快。
在实际应用中，我们往往会根据自己的需要权衡速度和准确性（时间和路程长短），来选择合适的启发函数。

> from:Barr & Feigenbaum 的 AI 手册 [Barr82, pg. 65，关于 A*]
> 其次，找到一个成本绝对最小的解决方案可能不如在中等长度的搜索中找到一个合理成本的解决方案重要。在这种情况下，人们可能更喜欢在大多数情况下更准确地评估节点但有时会高估到目标的距离的 h*，从而产生不可接受的算法。
> 以及 Dechter 和 Pearl [Dechter85] 的论文：
> 然而，已经发现维持 A* 的可接受性过于严格；它将启发式方法的选择限制在那些仅低估成本的方法上，并且它迫使 A* 花费不成比例的长时间来区分大致相等的候选人。因此，已经进行了几次尝试来执行具有不可接受的估计的 A*，同时限制次优的程度 并最大限度地减少高估可能导致的计算困难 。

2.21getHeu

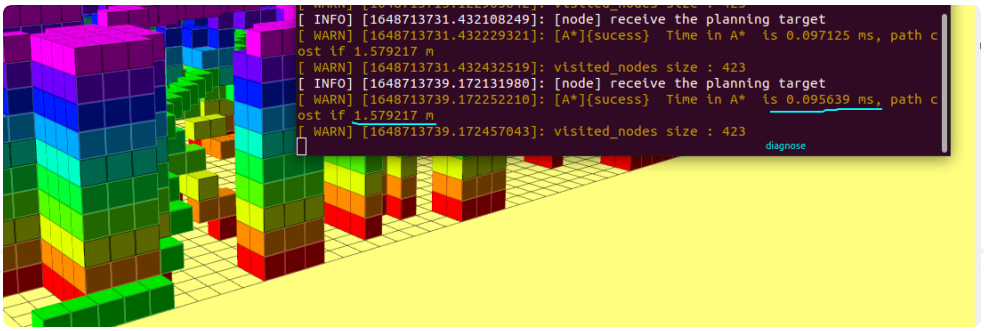
首先看官方给的三种启发函数的优劣比较：
在方形网格中：

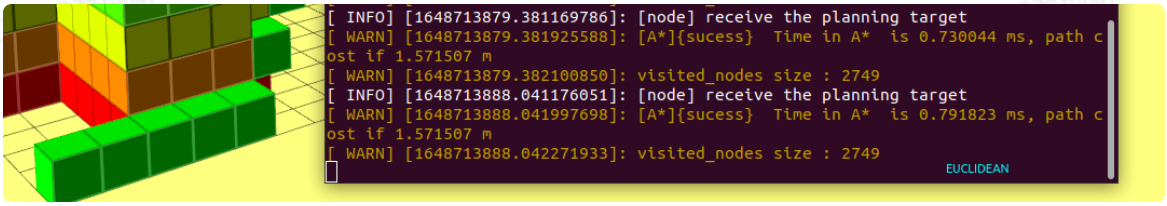
- 4方向移动，用Manhattan
- 8方向移动，用对角线距离
- 任意方向用欧式距离

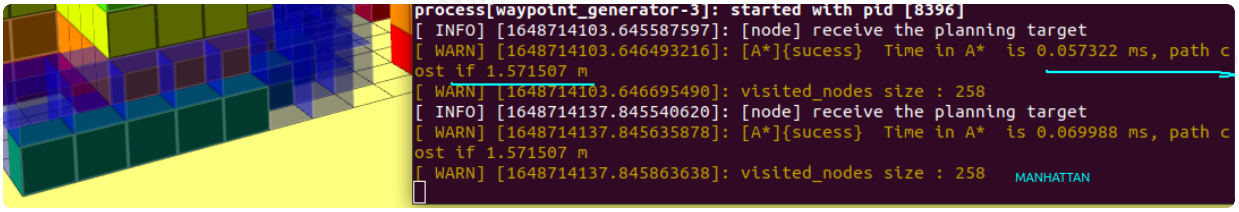
还有Octile距离：

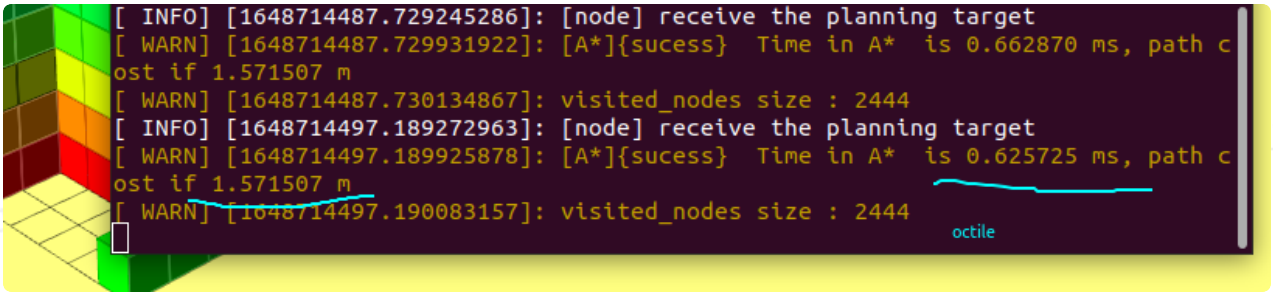
$$\sqrt{2}Min(abs(i.x-g.x),abs(i.y-j.y),abs(i.z-j.z))+Max(abs(i.x-g.x),abs(i.y-g.y),abs(i.z-g.z))$$

简单的看一下他们在源码中的比较：
(在固定地图以后设置统一终点)



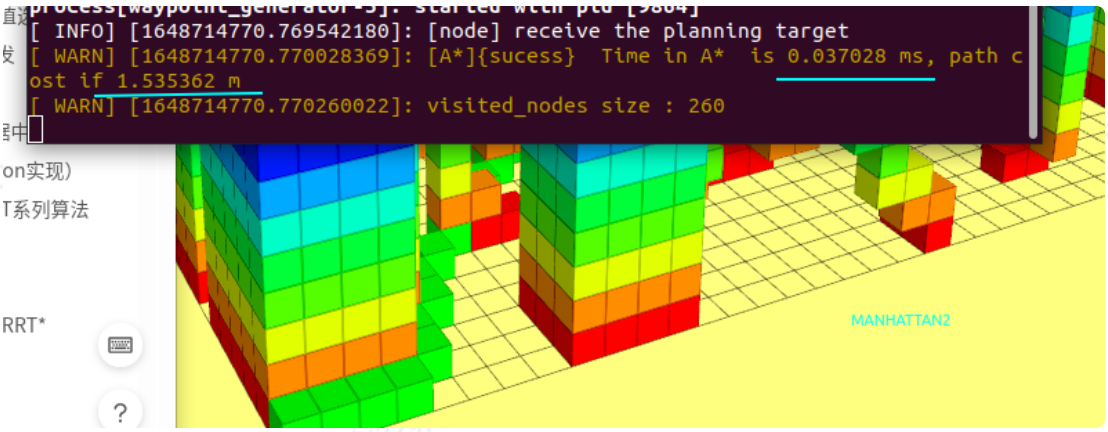




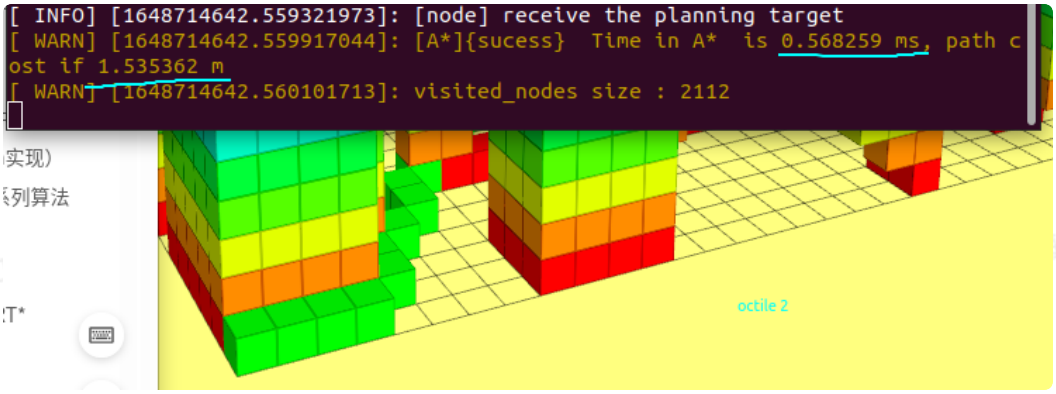


将

Manhattan和OCTILE再次比较：



哈工大威海创新平台 - 718-夏天怡(22970116)



在少量的测试里，

Manhattan奇迹地取得了最好的成绩。

`getHeu` 的优化思路:

1. 在原有的三种启发函数上,做数学上的修改
2. 局部欧氏距离
3. 根据实际需要,设计自己的启发函数

对于1,

二维情况:

C++ | 复制代码

```cpp
int estimateValue(Point goal,Point current)
{
    int dx = abs(goal.x-current.x);
    int dy = abs(goal.y-current.y);
    if(dx > dy)
        return 10*dx+5*dy;
    else
        return 10*dy+5*dx;
}
```

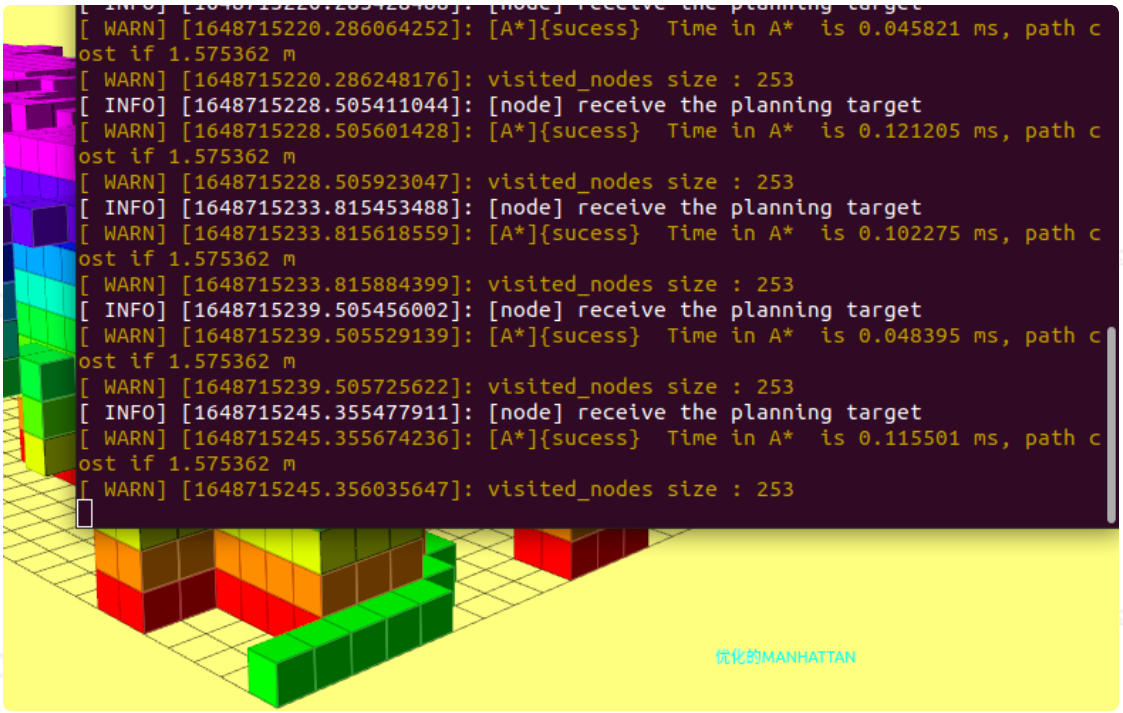
哈工大威海创新平台 - 718-夏天怡(22970116)

简化的三维情况：

C++ | 复制代码

```
double AstarPathFinder1::getHeu(GridNodePtr node1, GridNodePtr node2)
{
    double h;
    auto node1_coord = node1->coord;
    auto node2_coord = node2->coord;
    double dx = std::abs(node1_coord(0)-node2_coord(0));
    double dy = std::abs(node1_coord(1)-node2_coord(1));
    double dz = std::abs(node1_coord(2)-node2_coord(2));
    if(dx>dy)
    {
        h = 10*dx+5*dy;
    }
    else
        h = 5*dx+10*dy;
    h+=10*dz;
    return h;
}
```

这个dz前面的系数是可以调整的。



对于2,基于曼哈顿距离一般可以获得转弯较少的路径，但遇到障碍物时会出现局部绕行的缺点；欧氏距离一般遇到障碍物时不会出现局部绕行，但在路径规划当中会出现转弯较多的情况。 我考虑采用全局使用曼哈顿距离而遇到障碍物使用欧氏距离的方法。

C++ | 复制代码

```cpp
//首先要在bianliang.cpp里创建全局变量int obs = 0;
double AstarPathFinder::getHeu(GridNodePtr node1, GridNodePtr node2, int q)
{
    double h;
    auto node1_coord = node1->coord;
    auto node2_coord = node2->coord;

    // Heuristics 1: Manhattan
    if(!obs)
        h = std::abs(node1_coord(0) - node2_coord(0) ) +
         std::abs(node1_coord(1) - node2_coord(1) ) +
         std::abs(node1_coord(2) - node2_coord(2) );

    // Heuristics 2: Euclidean
    else
        h = std::sqrt(std::pow((node1_coord(0) - node2_coord(0)), 2 ) +
         std::pow((node1_coord(1) - node2_coord(1)), 2 ) +
         std::pow((node1_coord(2) - node2_coord(2)), 2 ));

    // Heuristics 3: Diagnol distance
    //double dx = std::abs(node1_coord(0) - node2_coord(0) );
    //double dy = std::abs(node1_coord(1) - node2_coord(1) );
    //double dz = std::abs(node1_coord(2) - node2_coord(2) );
    //double min_xyz = std::min({dx, dy, dz});
    //h = dx + dy + dz + (std::sqrt(3.0) -3) * min_xyz;

    return h;
}

inline void AstarPathFinder::AstarGetSucc(GridNodePtr currentPtr, vector<GridNodePtr> & neighborPtrSets, vector<double> & edgeCostSets)
{
    ~~~

    for(int i = -1;i <= 1;++i ){
        for(int j = -1;j <= 1;++j ){
            for(int k = -1;k <= 1;++k){
                if( i == 0 && j == 0 && k == 0)
                    continue;

                n_x = this_x + i;
                n_y = this_y + j;
                n_z = this_z + k;

```

```
                if( (n_x < 0) || (n_x > (GLX_SIZE - 1)) || (n_y < 0) ||
(n_y > (GLY_SIZE - 1) ) || (n_z < 0) || (n_z > (GLZ_SIZE - 1)))
                    continue;

                if(isOccupied(n_x, n_y, n_z))
                //{
                    //obs = 1;
                    //说明neighbour存在obs
                    continue;
                //}
                temp_ptr = GridNodeMap[n_x][n_y][n_z];

                if(temp_ptr->id == -1)
                    continue;

                n_coord = temp_ptr->coord;

                ~~~
                neighborPtrSets.push_back(temp_ptr);
                edgeCostSets.push_back(dist);

            }
        }
    }
}
void AstarPathFinder::AstarGraphSearch(Vector3d start_pt, Vector3d
end_pt)
{
    ~~~

    while ( !openSet.empty() ){

        ~~~
        //startPtr -> fScore = getHeu(startPtr,endPtr,0);
        AstarGetSucc(currentPtr, neighborPtrSets, edgeCostSets);

        for(int i = 0; i < (int)neighborPtrSets.size(); i++){

            neighborPtr = neighborPtrSets[i];
            if(neighborPtr -> id == 0){

                neighborPtr->gScore = currentPtr->gScore +
edgeCostSets[i];
                //neighborPtr->fScore = neighborPtr->gScore +
getHeu(neighborPtr,endPtr,obs);
                neighborPtr->cameFrom = currentPtr;

```

```
87                  openSet.insert(make_pair(neighborPtr -> fScore,
    neighborPtr));
88                  neighborPtr -> id = 1;
89                  continue;
90              }
91              else if(neighborPtr -> id == 1){
92                  if( neighborPtr->gScore > (currentPtr-
    >gScore+edgeCostSets[i]))
93                  {
94                      neighborPtr -> gScore = currentPtr -> gScore +
    edgeCostSets[i];
95                      //neighborPtr -> fScore = neighborPtr -> gScore +
    getHeu(neighborPtr,endPtr,obs);
96                      neighborPtr -> cameFrom = currentPtr;
97                  }
98
99                  continue;
100             }
101             else{
102                 continue;
103             }
104         }
105     }
106     ~~~
107 }
108
```

对于3,在我阅读一位学长的论文的时候,我看到了他设计的一个启发函数

$$h(\tilde{\mathbf{x}}_n) = ||x_N - x_{goal}||^2 + ||y_N - y_{goal}||^2 + ||\phi_N - \phi_{goal}||^2 + \lambda(1 - \cos(\theta_N - \theta_{goal})), \quad (15)$$

具体参数我就不介绍了,但是从中给了我很大的启发,我可以让 $h(n)$ 中包含机器人运动的自由度,去对机器人实际的运动特性和状态做出规划与限制。

然而仅仅通过优化 `getHeu` 得到的最终路径仍然会存在问题，由于计算出来的路线看起来过于"僵硬",我们往往不会要求机器人去这样走.对此,上文中我提到的theta*和lazy theta*算法就很有用了。

2.22getAngle

传统A*算法把机器人简化成一个质点，从而达到简化计算的目的，但是在实际应用中，这样会存在很大的弊端：A*忽略了机器人实体自身的运动特性，导致算法规划出的路径机器人执行难度较大，甚至无法执行

另外过多的转向需求（折点）也会使定位传感器的观测数据产生累计误差，这样会对机器人的定位造成误差甚至失败的情况，综合考虑我们引入角度约束来有效限制机器人的转向，尽管最后的路径可能不是最短的，但是却是代价更低的，于是我们实际评价函数改进成：

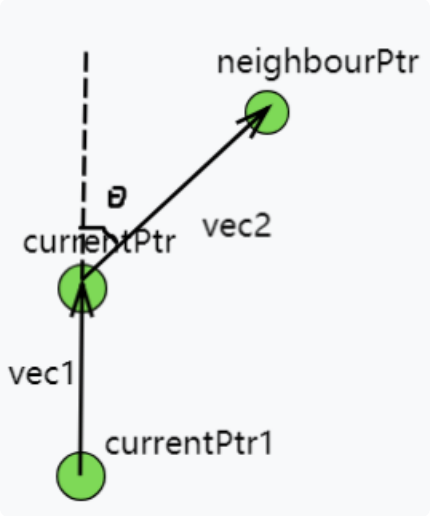
$$f(n) = g(n) + h(n) + r(n)$$

$r(n)$ 为转向代价函数，由 `getAngle()` 得到，通常是以下的计算方式：

$$r(n) = k * (\theta_n - \theta_{n-1})$$

其中 θ_n 是当前时刻朝向角， θ_{n-1} 是上一时刻机器人的朝向角。k越大，对角度的约束越明显，机器人更不易在一次选择中选择这条转弯过大的方式。

为了得出转向角我采用向量点乘的方式：



$$r = cos\theta = \frac{\vec{vec1} \cdot \vec{vec2}}{|vec1||vec2|}\ ,\ \theta = arccosr$$

哈工大威海创新平台 - 718-夏天怡(22970116)

C++ 复制代码

```cpp
//利用向量点乘求夹角
//dot
double AstarPathFinder::getAngle(Vector3d vec1,Vector3d vec2)
{
    double h1;
    //auto node1_coord = node1->coord;
    //auto node2_coord = node2->coord;
    h1 = vec1(0)*vec2(0)+vec1(1)*vec2(1)+vec1(2)*vec2(2);
    double h2,h3;
    h2 = std::sqrt(std::pow(vec1(0), 2 ) +
                   std::pow(vec1(1), 2 ) +
                   std::pow(vec1(2), 2 ));
    h3 = std::sqrt(std::pow(vec2(0), 2 ) +
                   std::pow(vec2(1), 2 ) +
                   std::pow(vec2(2), 2));
    double r = (double)h1/(h2*h3);
    r = std::acos(r);
    return r;
}

void AstarPathFinder::AstarGraphSearch(Vector3d start_pt, Vector3d
end_pt)
{

    ~~~
    //GridNodePtr currentPtr1 = NULL;
    //int t = 0;
    //Vector3i vec1 = 0;
    //Vector3i vec2 = 0;
    startPtr -> gScore = 0;
    startPtr -> fScore = getHeu(startPtr,endPtr);
    startPtr -> id = 1;
    startPtr -> coord = start_pt;
    openSet.insert( make_pair(startPtr -> fScore, startPtr) );

    GridNodeMap[start_idx[0]][start_idx[1]][start_idx[2]] -> id = 1;
    ~~~

    while ( !openSet.empty() ){

        int x = openSet.begin()->second->index(0);
        int y = openSet.begin()->second->index(1);
        int z = openSet.begin()->second->index(2);
        openSet.erase(openSet.begin());
        currentPtr = GridNodeMap[x][y][z];
```

```
45
46          /*
47          //主要是对第一个节点的一些小处理
48          if(t)
49          {
50              currentPtr1 = GridNodeMap[x][y][z];
51              t--;
52          }
53          if(!t)
54              vec1 = currentPtr->index - currentPtr1->index;
55          */
56
57          if(currentPtr->id == -1)
58              continue;
59
60          currentPtr->id = -1;
61
62          AstarGetSucc(currentPtr, neighborPtrSets, edgeCostSets);
63
64          for(int i = 0; i < (int)neighborPtrSets.size(); i++){
65
66              neighborPtr = neighborPtrSets[i];
67              //vec2 = neighborPtr->index - currentPtr->index;
68              //double r = getAngle(vec2,vec1);
69              if(neighborPtr -> id == 0){
70
71                  neighborPtr->gScore = currentPtr->gScore +
    edgeCostSets[i];
72                  //neighborPtr->fScore = neighborPtr->gScore +
    getHeu(neighborPtr,endPtr) + r;
73                  neighborPtr->cameFrom = currentPtr;
74
75                  openSet.insert(make_pair(neighborPtr -> fScore,
    neighborPtr));
76                  neighborPtr -> id = 1;
77                  continue;
78              }
79              else if(neighborPtr -> id == 1){
80                  if( neighborPtr->gScore > (currentPtr-
    >gScore+edgeCostSets[i]))
81                  {
82                      neighborPtr -> gScore = currentPtr -> gScore +
    edgeCostSets[i];
83                      //neighborPtr -> fScore = neighborPtr -> gScore +
    getHeu(neighborPtr,endPtr) + r;
84                      neighborPtr -> cameFrom = currentPtr;
85                  }
86
```

```
                    continue;
                }
                else{
                    continue;
                }
            }
            //currentPtr1 = GridNodeMap[x][y][z];
        }

    }

```

2.23添加权重w

$f(x)=g(x)+w*h(x)$ 称其为weighted A*。

使用最优性换取速度，比A*算法快几个数量级，研究的重点变为寻找具有浅局部最小值的启发值，最终得到的路径是suboptimal的.

而对于权重的处理，已有国内外的学者对其做了不同的优化，

我尝试了几种不同的优化方式：

1. 简单地令w为一些大于一的数
2. e^h
3. fn=(1−λ)*gn+λhn(λ与hn有关)
4. 在AAAI 2021文章（Necessary and Sufficient Conditions for Avoiding Reopenings in Best First Suboptimal Search with General Bounding Functions）展示了两种方法：

- pwXU，在终点附近降低权重

$[g<(2w-1)h]:f=g/(2w-1)+h;$

$[g\geq(2w-1)h]:f=(g+h)/w$

- pwXD，在终点附近增加权重

$[h>g]:f=g+h;$

$[h\leq g]:f=(g+(2w-1)h)/w$

5. 膨胀式A*

迭代运行加权A*，同时不断减小 w 使其次优解不断向最优解靠近。这一简单想法就可以得到一系列带有次优性因子（即膨胀因子 w ，用来衡量前文所说的次优解的优劣）的次优解。但是只有这样的话仍然无法解决问题，因为简单的重复运行加权A*会有大量已经被计算过的节点被重复计算，这会浪费大量的计算时间。

针对这个问题，国外已有学者做出了较为详尽的分析，这里我参考了由Carnegie Mellon University 提出的ARA*算

法.https://proceedings.neurips.cc/paper/2003/file/ee8fe9093fbbb687bef15a38facc44d2-Paper.pdf （其中有可行性证明）

ARA*在weighted A*的基础上，改进了w的取值方式，使得w会越来越接近1，在时间允许的范围内，w会变成1，时间不够时，w会尽可能接近1。之所以要接近1，是因为我们知道拥有合理的启发函数的A*算法在理论上是最优的（路径上），而w退化成1时便是A*。

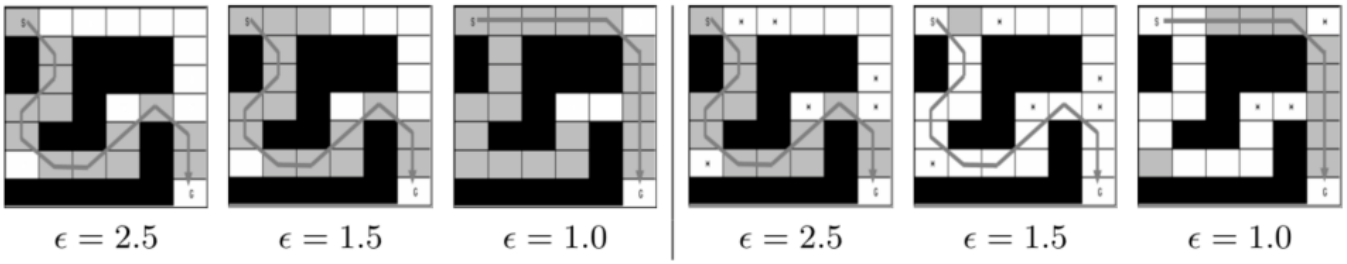


Figure 2: Left three columns: A* searches with decreasing ϵ. Right three columns: the corresponding ARA* search iterations.

from essay

下面是ARA*执行的思路：

1. 先选择一个较大的w，利用Improvepath()快速地规划出一条suboptimal的路线
2. 从第一步我们会得到一个openset、incons和closeset，然后我们将openset和incons合并可以得到一个包含所有inconsistent状态节点的容器。
3. 然后在主寻路函数里将w降低，在包含所有Inconsistent状态节点的容器里进行搜索，规划出一条相对更优的路径
4. 在循环中重复2，3步，直到路径收敛（总代价没有明显变化）或者w降低为1停止
5. 然后返回路径

代码:

Improvepath()　　C++　　复制代码

```
void AstarPathFinder::ImprovePath(GridNodePtr Ptr)
{
    //search for a suboptimal path
    std::vector<Vector3d> visitedupdate;
    visitedupdate.clear();

    GridNodePtr currentPtr  = NULL;
    GridNodePtr neighborPtr = NULL;

    vector<GridNodePtr> neighborPtrSets;
    vector<double> edgeCostSets;
    ROS_WARN("enter in improvepath");
    while(1)
    {
        //get p-node with smallest f
        ROS_WARN("OPENSETsize:%d",openSet.size());
        GridNodePtr pp= smallest_f(openSet);
        int x = pp->index(0);
        int y = pp->index(1);
        int z = pp->index(2);
        if((Ptr->fScore) <= (pp->fScore))
        {
            ROS_WARN("PATHFouND, end->fscore:%lf,p-
>fscore:%lf",getfvalue(Ptr,Ptr),pp->fScore);
            break;
        }
        currentPtr = GridNodeMap[x][y][z];
        for(auto itr =openSet.begin();itr != openSet.end();itr++)
        {
            //ROS_WARN("1");
            if(pp == *itr){
                ROS_WARN("2");
                itr = openSet.erase(itr);
                break;
            }
        }
        ROS_WARN("1.1");
        closeset.push_back(currentPtr);
        currentPtr->id = -1;

        flag = 1;//for test
        AstarGetSucc(currentPtr, neighborPtrSets, edgeCostSets);
        for(int i = 0; i < (int)neighborPtrSets.size(); i++)
        {
            //ROS_WARN("enter in for loop for %d times",i);
```

```
45              neighborPtr = neighborPtrSets[i];
46              if(neighborPtr==Ptr) {
47                  ROS_WARN("YOU FIND THE GOAL!");
48                  break;
49              }
50
51              if(neighborPtr->id == 0 || neighborPtr->gScore > (currentPtr-
    >gScore+edgeCostSets[i]))
52              {
53                  ROS_WARN("neiboorid:%d",neighborPtr->id);
54                  neighborPtr -> gScore = currentPtr -> gScore +
    edgeCostSets[i];
55                  neighborPtr -> cameFrom = currentPtr;
56                  visitedupdate.push_back(neighborPtr->coord);
57                  neighborPtr->id = 1;
58
59                  vector<GridNodePtr>::iterator it1 =
    find(closeset.begin(), closeset.end(), neighborPtr);
60
61                  if(it1==closeset.end()) {
62                      ROS_WARN("if");
63                      openSet.push_back(neighborPtr);
64                      ROS_WARN("openSet.push success");
65                      //neighborPtr->fScore = neighborPtr->gScore +
    getHeu(neighborPtr, endPtr);
66                      neighborPtr->fScore = getfvalue(neighborPtr,Ptr);
67                  }
68                  else
69                      ROS_WARN("ELSE");
70                      incons.insert((make_pair(0.0,neighborPtr)));
71
72              }
73
74          }
75      }
76      ROS_WARN("leave the loop!");
77      for(auto it = visitedupdate.begin();it!=visitedupdate.end();it++)
78          visited.push_back(*it);
79  }
```

哈工大威海创新平台 - 718-夏天怡(22970116)

smallest_f()

C++ 复制代码

```
GridNodePtr AstarPathFinder::smallest_f(vector<GridNodePtr> set)
{
    ROS_WARN("enter in smallest_f");
    ROS_WARN("SET.BEGIN:%d",*set.begin());
    GridNodePtr p=*set.begin();

    //ROS_WARN("openset.begin: %d",p);
    for(auto it1 = set.begin();it1 != set.end();it1++){
        if((*it1)->fScore<p->fScore) {
            p = *it1;
        }
    }
    return p;
}
```

哈工大威海创新平台 - 718-夏天怡(22970116)

AstarGraphSearch()　　C++　　复制代码

```cpp
//为新增部分
void AstarPathFinder::AstarGraphSearch(Vector3d start_pt, Vector3d end_pt)
{
    ros::Time time_1 = ros::Time::now();
    //path.clear();
    //visited.clear();
    //closeset.clear();
    //incons.clear();
    //openSet.clear();
    Vector3i start_idx = coord2gridIndex(start_pt);
    Vector3i end_idx   = coord2gridIndex(end_pt);

    start_pt = gridIndex2coord(start_idx);
    end_pt   = gridIndex2coord(end_idx);

    GridNodePtr startPtr = new GridNode(start_idx, start_pt);
    GridNodePtr endPtr = new GridNode(end_idx, end_pt);
    startPtr -> gScore = 0;
    startPtr -> fScore = getHeu(startPtr,endPtr);
    startPtr -> id = 1;
    ROS_WARN("ID:%d",startPtr->id);
    startPtr -> coord = start_pt;
    startPtr -> cameFrom = startPtr;

    openSet.push_back(startPtr);
    endPtr ->gScore=99999.0;//设置一个很大的数
    endPtr ->fScore=99999.0;
    GridNodeMap[start_idx[0]][start_idx[1]][start_idx[2]] -> id = 1;

    //ImprovePath(endPtr);

    /*vector<Vector3d> ppath= getPath(endPtr);
    for(auto it = ppath.begin();it!=ppath.end();it++)
        path.push_back(*it);*/

    /*while(update_w(endPtr)>1) {
        ROS_WARN("enter in while");
        w -= 0.4;

        for (auto iti = incons.begin(); iti != incons.end(); iti++) {
            vector<GridNodePtr>::iterator it = find(openSet.begin(), openSet.end(), iti->second);
            if (it == openSet.end())
                openSet.push_back(iti->second);
```

```
        }
        for (auto iti = openSet.begin(); iti != openSet.end(); iti++) {
             (*iti)->fScore = getfvalue(*iti,endPtr);
        }

        incons.clear();
        closeset.clear();

        ImprovePath(endPtr);

        for (auto it = getPath(endPtr).begin(); it < getPath(endPtr).end(); it++) {
            path.push_back(*it);
        }

       }*/
      ros::Time time_2 = ros::Time::now();
      //w = 1.5;
      ROS_WARN("[A*]{sucess}  Time in A*  is %f ms, path cost if %f m,test %d", (time_2 - time_1).toSec() * 1000.0, endPtr->gScore * resolution,flag );
      return;
}

    ros::Time time_2 = ros::Time::now();

    if((time_2 - time_1).toSec() > 0.1)
        ROS_WARN("Time consume in Astar path finding is %f", (time_2 - time_1).toSec() );

}
```

getfvalue C++ 复制代码

```
double AstarPathFinder::getfvalue(GridNodePtr node1,GridNodePtr node2)
{
    return node1->gScore+w* getHeu(node1,node2);
}
```

2.24对于相等f值选择的处理

方法有：

1. h(n)->(1+p)*h(n)，就是添加权重
2. 直接比较h(n)，选择h(n)较小的节点

3. 将确定性随机数添加到启发式或边缘成本，一般会计算坐标的hash值
4. 选择在起点和终点连线上的点（或更接近）

- 计算开始到目标向量和当前点到目标向量之间的向量叉积,由于两个三维向量叉乘得到的向量也是三维的，我去三个坐标的和来近似叉乘的结果。当这些向量不对齐时，叉积会更大。结果是该代码将略微偏爱从起点到目标的直线路径。这个方法在障碍物较少的情况下得到的路径会十分好看，探索的节点也很少。

方法4 C++ 复制代码

```
dx1 = current.x - end.x
dy1 = current.y - end.y
dz1 = current.z - end.z
dx2 = start.x - end.x
dy2 = start.y - end.y
dz2 = start.z - end.z
cross = abs(dy1*dz2-dz1*dy2 + (dx1*dz2-dz1*dx2) + dx1*dy2-dy1*dx2)
heuristic += cross*e
e = 0.0001//自己取值,调整
```

5. 还可以通过对转弯做一些限制来打破平衡（转2.22）

2.3从数据结构出发

2.31A*

因为需要存储节点的各种信息，因此选择一个合适的数据结构（容器）去存储，是很有必要的。而数据结构的选择不仅取决于操作，还取决于每个操作运行的次数。

在本次A*算法中我们需要用到的情况：

1. openSet–>存节点指针和f值
2. fopenSet(这是我自己创建的查找容器，只存储和openSet里完全一样的节点指针，用于查找，判断终止)（双向A*中）
3. 有些A*代码中创建了closeset列表，我觉得我们的代码用id=-1来表示这一点十分明智地，因为closeset只起到单一的判断作用，所以直接将其内化成节点的属性，这样在每次判断的时候无需遍历整个closeset容器，这样会节约很多时间和空间。

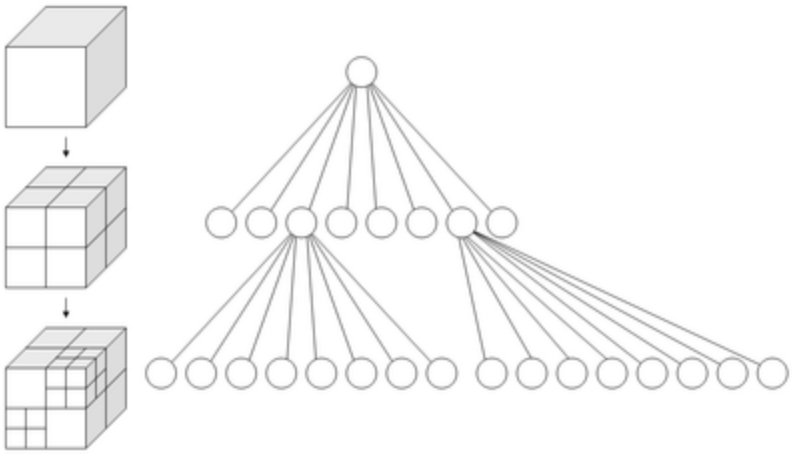
而我们主要执行的操作有：

1. 取出并删除最佳节点
2. 插入邻居节点
3. 根据f值对节点进行排序

官方给的代码使用了 `multimap` 这个stl容器作为openset，也是十分好的，因为它内部是建立了一个平衡二叉树（红黑树），对于openset我们关心其中f值最小的节点，因此用它取出和删除(1)会很方便，另外内部的结构也为自动按照key值（fscore)来排序存(3)储，也为我们实现带来了方便。同时，官方用ptr->id = -1来表示节点位于closeset中，也是十分好的，因为它省去了遍历closeset查找ptr是否位于其中的操作，这样有效的降低了时间复杂度。
链表(list)的时间复杂度：成员遍历 $O(F)$, 移除和尾部插入 $O(1)$ ，增加优先度 $O(F)$
二叉堆的时间复杂度：查找 $O(F)$ ，插入 $O(logF)$ ，删除最优 $O(logF)$ ，增加优先度 $O(logF)$
哈希表也是一个可以被考虑使用的数据结构，它的成员查找，插入，移除，提升优先等级的时间复杂度都是 $O(1)$ ，不过它不适合进行排序，并且为相应的节点去找一个合适的hash函数也并不容易。

2.32在点云数据中建立空间索引

对于三维地图，通过观察可以发现障碍物只是其中一小部分，我们可以使用一些特殊的数据结构来存储，比如八叉树(octree)。



from Wiki

八叉树（octree）是一种基于树形层次的数据结构， 其定义是：若不为空树的话，树中任一节点的子节点恰好只会有八个或零个。八叉树充分利用了形体在空上的相关性，因此，它所占用的存贮空间要比三维体素阵列的少。其主要优点：管理方便，搜索某一个小方块的时候，能很方便的使用二分法查找，当深度到一定层次 以后，基本可以拟合所有的三维模型。
在未来的路径规划中，我可以尝试不同的空间索引，来节约存储空间，也方便查询。

3.拓展部分（用python实现）

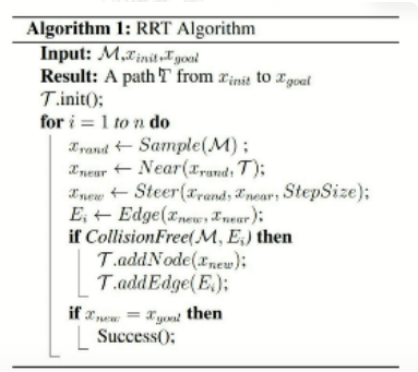
为了更好地在线下赛场进行路径规划，我还学习了以下内容为线下赛做准备，同时丰富自己专业知识。

而本次线上赛选择的rviz的3d栅格坐标显示在一定程度上对我们使用的路径规划算法有所限制，比如我们难以通过数学方法对最后的路线进行平滑等处理，采样或者智能算法也不便于使用，而在实际应用中它们又是被广泛使用的所以我也对它们进行了学习与研究。
同时也对这些代码利用python来实现，为什么用python呢，因为在官方给的环境里路径规划的再利用性不高，不方便未来的移植和部署，所以我直接用python完成，这样的代码比较清晰，并且便于未来的移植。我对代码的开源部分进行了学习，并且在本地环境进行了复现，代码与效果也在下文给出。

3.1基于采样的RRT系列算法

3.12RRT

快速扩展随机数算法(Rapidly Exploring Random Tree)，不像Dijkstra算法那样基于栅格一步步扩展，而是从根节点长成一棵树，当树枝长到终点时，我们就得到了一条到终点的路径。
伪代码：



流程如下：

1. 先利用采样函数得到Xrand（在空间中随机选择一个点）
2. 然后寻找距离这个Xrand距离最近的节点
3. Steer进行树的具体的生长，将Xrand和Xnear连接，作为树生长的具体方向，设置StepSize（步长），作为树枝的长度，然后会产生Xnew，此时Xnear和Xnew为所有的节点，本次循环结束
4. 然后继续重复上述过程：
5. 1~4中我们还需要判断是否穿越了障碍物：若穿过则抛弃此次生长
6. 经过多次采样，会越来越靠近终点，所以要设置提前终止的条件，在每次产生Xnew的时候，会把它和终点进行连线，判断它们之间的距离是否比步长要小，且是否没有障碍物。
7. 上述过程有一个小技巧，我们以一定概率（5%～10%）（为了防止当前点与终点之间存在障碍物而导致经过障碍物）选择终点作为采样点，这样就可以导向终点，加快向终点的生长。

对应代码部分：
先定义RRT类和Node类

关键代码 C++ 复制代码

```
class Node:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.cost = 0.0
        self.parent = None
class RRT:
    def __init__(self, samplemap, obslist, itertimes=300, samplerate=20, growdis=2):
        self.begin = None
        self.end = None
        self.obs_list = obslist
        self.node_list = None
        self.sample_min = samplemap[0]
        self.sample_max = samplemap[1]
        self.iter_times = itertimes
        self.grow_dis = growdis
        self.sample_rate = samplerate

    def rrt_basic(self, begin, end, visualization=True):
        self.begin = Node(begin[0], begin[1])
        self.end = Node(end[0], end[1])
        #初始根节点放入node_list
        self.node_list = [self.begin]
        path = None

        for i in range(self.iter_times):
            #进入采样函数，得到Xrand(在空间中随机选择一个点)
            rnd = self.sample()
            # 在node_list中寻找距离这个Xrand距离最近的节点
            n_ind = self.get_nearestnode(self.node_list, rnd)
            # 得到最近节点Xnear
            nearestNode = self.node_list[n_ind]
            #树的具体生长
            #方向角theta
            theta = math.atan2(rnd[1] - nearestNode.y, rnd[0] - nearestNode.x)
            newNode = self.get_new_node(theta, n_ind, nearestNode)
            #检查新选中的节点和最近节点之间是否有障碍物
            noobs = self.obsdetect(newNode.x, newNode.y, nearestNode.x, nearestNode.y)
            #若无，则:
            if noobs:
                #将新选中的节点添加到node_list中
                self.node_list.append(newNode)
```

```
                #关于可视化
                if visualization:
                    self.draw_graph(newNode, path)
                #对新节点做终点检测（判断是否停止生长）
                if self.enddetect(newNode):
                    #如果二者之间没有障碍，则:
                    if self.obsdetect(newNode.x, newNode.y,
                                                    self.end.x,
self.end.y):
                        #
                        lastIndex = len(self.node_list) - 1
                        #从lastIndex中得到path
                        path = self.getpath(lastIndex)
                        #计算path长
                        pathLen = self.get_path_len(path)
                        print("current path length: {}".format(pathLen))

                        if visualization:
                            self.draw_graph(newNode, path)
                        return path

```

rrt里用到的一些函数

采样函数sample C++ 复制代码

```
    def sample(self):#在地图范围采点，并且给予一定的sample_rate，选择终点，来引导采样方向
        if random.randint(0, 100) < self.sample_rate:#如果产生的0~100的随机数小于sample_rate
            rnd = [self.end.x, self.end.y]
        else:
            rnd = [random.uniform(self.sample_min, self.sample_max), random.uniform(self.sample_min, self.sample_max)]
        return rnd

```

找最近的根节点get_nearestnodes C++ 复制代码

```
    @staticmethod
    def get_nearestnode(nodes, rnd):
        dis = [(node.x - rnd[0]) ** 2 + (node.y - rnd[1]) ** 2 for node
 in nodes]
        minIndex = dis.index(min(dis))
        return minIndex

```

生长grow_node C++ 复制代码

```
    def grow_node(self, theta, n_ind, nearestNode):
        newNode = copy.deepcopy(nearestNode)

        newNode.x += self.grow_dis * math.cos(theta)
        newNode.y += self.grow_dis * math.sin(theta)

        newNode.cost += self.grow_dis
        newNode.parent = n_ind
        return newNode
```

obstaclesdetect C++ 复制代码

```
    def obsdetect(self, x1, y1, x2, y2):
        for (ox, oy, size) in self.obs_list:
            dd = self.distance(
                np.array([x1, y1]),
                np.array([x2, y2]),
                np.array([ox, oy]))
            if dd <= size ** 2:
                return False  # there is obstacle
        return True

```

哈工大威海创新平台 - 718-夏天怡(22970116)

计算distance C++ 复制代码

```
    @staticmethod
    def distance(v, w, p):
        if np.array_equal(v, w):
            return (p - v).dot(p - v)
        l2 = (w - v).dot(w - v)
        t = max(0, min(1, (p - v).dot(w - v) / l2))
        projection = v + t * (w - v)
        return (p - projection).dot(p - projection)
```

设置提前终止的条件 C++ 复制代码

```
    def enddetect(self, node):
        d = self.pathcost(node, self.end)
        if d < self.grow_dis:
            return True
        return False
```

哈工大威海创新平台 - 718-夏天怡(22970116)

一些关于path的函数 C++ 复制代码

```
    def pathcost(self, node1, node2):
        return math.sqrt((node1.x - node2.x) ** 2 + (node1.y - node2.y)
** 2)

    def getpath(self, lastIndex):
        path = [[self.end.x, self.end.y]]
        while self.node_list[lastIndex].parent is not None:
            node = self.node_list[lastIndex]
            path.append([node.x, node.y])
            lastIndex = node.parent
        path.append([self.begin.x, self.begin.y])
        return path

    def get_path_len(self, path):
        pathLen = 0
        for i in range(1, len(path)):
            nodex1 = path[i][0]
            nodey1 = path[i][1]
            nodex2 = path[i - 1][0]
            nodey2 = path[i - 1][1]
            pathLen += math.sqrt((nodex1 - nodex2)
                                 ** 2 + (nodey1 - nodey2) ** 2)
        return pathLen

```

哈工大威海创新平台 - 718-夏天怡(22970116)

地图和障碍绘制 C++ 复制代码

```
    def draw_graph(self, rnd=None, path=None):
        plt.clf()
        plt.gcf().canvas.mpl_connect(
            'key_release_event',
            lambda event: [exit(0) if event.key == 'escape' else None])
        if rnd is not None:
            plt.plot(rnd.x, rnd.y, "^b")

        for node in self.node_list:
            if node.parent is not None:
                if node.x or node.y is not None:
                    plt.plot([node.x, self.node_list[node.parent].x], [
                        node.y, self.node_list[node.parent].y], "-g")

        for (ox, oy, size) in self.obs_list:
            plt.plot(ox, oy, "oc",ms=30*size)

        plt.plot(self.begin.x, self.begin.y, "xr")
        plt.plot(self.end.x, self.end.y, "xr")

        if path is not None:
            plt.plot([x for (x, y) in path], [y for (x, y) in path], '-
r')

        plt.axis([-2, 18, -2, 15])
        plt.grid(True)
        plt.pause(0.01)

```

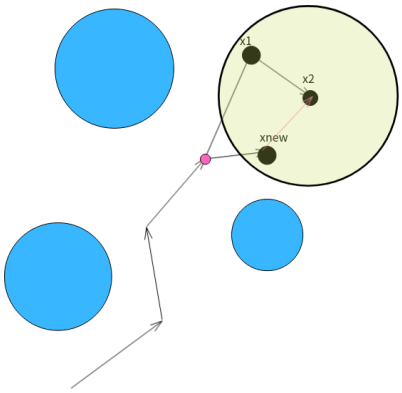
3.13RRT*

原理：

由于RRT产生的路径比较糟糕，所以提出了RRT*以找到一个更优的路径

对一个生长出去的节点，我们在一定范围内对它重新选择父节点（看看是否有更优的选择），而这个范围是我们自己指定的，

哈工大威海创新平台 - 718-夏天怡(22970116)



如图所示，xnew作为x2的父节点比x1作为父节点有更短的路径，也就是重联后更好的结果。根据有关论文的证明，每次对范围内的节点重新连接，可以找到一条渐近最优的路径。（经过多轮迭代）

代码实现：

相较于RRT算法，只需要在上述部分做出相应的改进即可。

哈工大威海创新平台 - 718-夏天怡(22970116)

C++ 复制代码

```
##表示新做出修改的部分
class RRT:
    def __init__(self, samplemap, obslist, itertimes=300, samplerate=20, growdis=2):
        self.begin = None
        self.end = None
        self.obs_list = obslist
        self.node_list = None
        self.sample_min = samplemap[0]
        self.sample_max = samplemap[1]
        self.iter_times = itertimes
        self.grow_dis = growdis
        self.sample_rate = samplerate

    def rrt_star(self, begin, end, visualization=True):
        self.begin = Node(begin[0], begin[1])
        self.end = Node(end[0], end[1])
        #初始根节点放入node_list
        self.node_list = [self.begin]
        path = None

        ##previouslength = float('inf')
        for i in range(self.iter_times):
            #进入采样函数，得到Xrand(在空间中随机选择一个点)
            rnd = self.sample()
            # 在node_list中寻找距离这个Xrand距离最近的节点
            n_ind = self.get_nearestnode(self.node_list, rnd)
            # 得到最近节点Xnear
            nearestNode = self.node_list[n_ind]
            #树的具体生长
            #方向角theta
            theta = math.atan2(rnd[1] - nearestNode.y, rnd[0] - nearestNode.x)
            newNode = self.get_new_node(theta, n_ind, nearestNode)
            #检查新选中的节点和最近节点之间是否有障碍物
            noobs = self.obsdetect(newNode.x, newNode.y, nearestNode.x, nearestNode.y)
            #若无，则:
            if noobs:
                #在指定范围内选择可能会被重连的节点
                ##nearinds = self.find_near(newNode)
                ##newNode = self.choose_parent(newNode, nearinds)

                #将新选中的节点添加到node_list中
                self.node_list.append(newNode)
```

```
43
44                  #重连
45                  ##self.rewire(newNode, path)
46
47                  #关于可视化
48                  if visualization:
49                      self.draw_graph(newNode, path)
50                  #对新节点做终点检测（判断是否停止生长）
51                  if self.enddetect(newNode):
52                      #如果二者之间没有障碍，则：
53                      if self.obsdetect(newNode.x, newNode.y,
54                                                      self.end.x,
    self.end.y):
55                          #
56                          lastIndex = len(self.node_list) - 1
57                          #从lastIndex中得到path
58                          temp_path = self.getpath(lastIndex)
59                          #计算path长
60                          temp_pathLen = self.get_path_len(temp_path)
61                          print("current path length:
    {}".format(temp_pathLen))
62
63                          ##delete if visualization:
64                          ##delete    self.draw_graph(newNode, path)
65
66                          ##if previouslength > temp_pathLen:
67                          ##    path = temp_path
68                          ##    previouslength = temp_pathLen
69                          ##    print("current pathlength:
    {}".format(temp_pathLen))
70          return path
```

新增函数有：

哈工大威海创新平台 - 718-夏天怡(22970116)

##find_near C++ 复制代码

```
    def find_near(self, newNode):
        n_node = len(self.node_list)
        #自己设置的查找半径
        r = 50.0 * math.sqrt((math.log(n_node) / n_node))
        d_list = [(node.x - newNode.x) ** 2 + (node.y - newNode.y) ** 2
                  for node in self.node_list]
        near_inds = [d_list.index(i) for i in d_list if i <= r ** 2]
        return near_inds

```

##choose_parent C++ 复制代码

```
    def choose_parent(self, newNode, nearInds):
        if len(nearInds) == 0:
            return newNode

        dList = []
        for i in nearInds:
            dx = newNode.x - self.node_list[i].x
            dy = newNode.y - self.node_list[i].y
            d = math.hypot(dx, dy)
            theta = math.atan2(dy, dx)
            if self.obsdetect1(self.node_list[i], theta, d):
                dList.append(self.node_list[i].cost + d)
            else:
                dList.append(float('inf'))

        minCost = min(dList)
        minInd = nearInds[dList.index(minCost)]

        if minCost == float('inf'):
            print("min cost is inf")
            return newNode

        newNode.cost = minCost
        newNode.parent = minInd

        return newNode
```

##rewire C++ 复制代码

```
    def rewire(self, newNode, nearInds):
        n_node = len(self.node_list)
        for i in nearInds:
            nearNode = self.node_list[i]

            d = math.sqrt((nearNode.x - newNode.x) ** 2
                          + (nearNode.y - newNode.y) ** 2)

            s_cost = newNode.cost + d

            if nearNode.cost > s_cost:
                theta = math.atan2(newNode.y - nearNode.y,
                                   newNode.x - nearNode.x)
                if self.obsdetect1(nearNode, theta, d):
                    nearNode.parent = n_node - 1
                    nearNode.cost = s_cost
```

##obsdetect1 C++ 复制代码

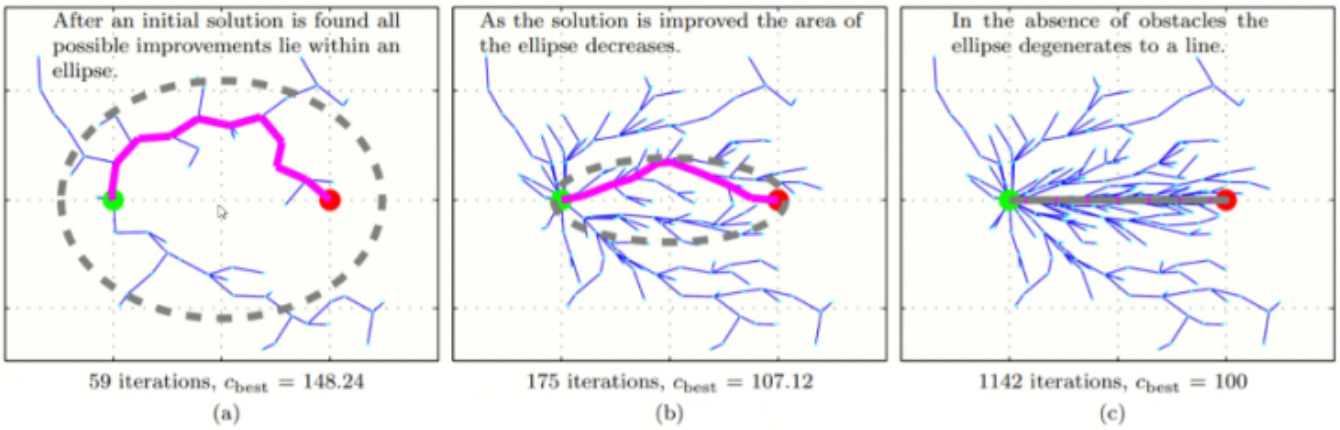
```
    def obsdetect1(self, nearNode, theta, d):
        tmpNode = copy.deepcopy(nearNode)
        end_x = tmpNode.x + math.cos(theta) * d
        end_y = tmpNode.y + math.sin(theta) * d
        return self.obsdetect(tmpNode.x, tmpNode.y, end_x, end_y)
```

3.14informed RRT*

由于RRT*是需要多次迭代从而得到一条渐近最优的路径，那么在多次迭代的过程中我们很自然的能够想到是否通过增加某种约束来增快搜索的速度呢?

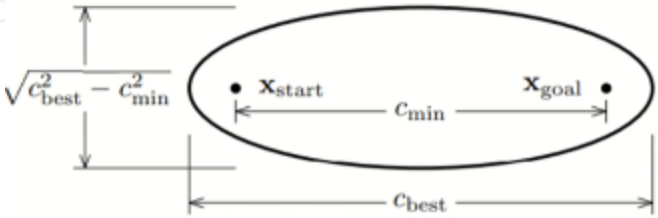
哈工大威海创新平台 - 718-夏天怡(22970116)

通过阅读论文和相关视频介绍



Algorithm 2: $\texttt{Sample}\left(\mathbf{x}_{start}, \mathbf{x}_{goal}, c_{max}\right)$

```
if c_max < ∞ then
    c_min ← ||x_goal − x_start||_2;
    x_centre ← (x_start + x_goal) /2;
    C ← RotationToWorldFrame (x_start, x_goal);
    r_1 ← c_max/2;
    {r_i}_{i=2,...,n} ← (sqrt(c_max^2 − c_min^2)) /2;
    L ← diag {r_1, r_2, ..., r_n};
    x_ball ← SampleUnitNBall;
    x_rand ← (CLx_ball + x_centre) ∩ X;
else
    x_rand ~ U(X);
return x_rand;
```



我们通过不断增大以起点和终点为焦点的椭圆的离心率来缩小椭圆的面积从而缩小采样的范围。

效果如视频。

2022-03-31 05-12-37.mp4

3.15ompl库

OMPL，一个开放的运动规划库，由许多基于采样的先进的运动规划算法组成。

它分为：

- Geometric planners：考虑系统的几何约束和运动学约束，一般机械臂使用这类规划器就好；

    - 综合查询方法
        - 基于概率路线图PRM(Probabilistic Roadmap Method)。PRM先构建路线图，再通过合适的图搜索算法（如A*）得到可行路径。
        - LazyPRM：利用延迟碰撞检测的思想来减少不必要碰撞检测的次数，从而提高规划效率；PRM*：可以搜索最优路径
    - 单一查询方法
        - 基于快速扩展随机树RRT(Rapidly-exploring Random Trees)。与PRM不同的是,RRT使用边搜索边采样，并构建地图的方法得到可行路径。
        - RRT：概率完备、非渐进最优；RRTStar：概率完备、渐进最优；BIRRT：双向RRT搜索，概率完备、非渐进最优
- Control-based planners：如果考虑的系统受到差分约束，则使用该类规划器。

这样的话，利用OMPL在ROS中便可以很容易实现RRT系列算法。

3.2局部路径优化

在ai航项目组的空间里听了几场16届优秀团队的经验分享，其中就有学长提到了DWA和 ，因此我打算借此机会也学习一下。

3.21DWA

机器人在经过全局规划得到一个通向终点的路径以后，会进行局部路径规划得到它在局部应该作出的行动策略，而ROS中主要是使用DWA算法。在ROS中每当move_base处于规划状态就调用DWA算法计算出一条最佳的速度指令，发送给机器人运动底盘执行。

原理：

在速度（v,w）空间中采样得到多组速度，然后模拟得出机器人未来的运动轨迹，再利用一个评价函数对这些路径打分，最后得将得出的最优速度值发给下位机让机器人去执行。

具体操作：

1. 对机器人速度进行约束限制，形成动态窗口进行速度采样；

2. 根据速度采样点求出待评价轨迹，最大化评价函数选取最优速度命令。

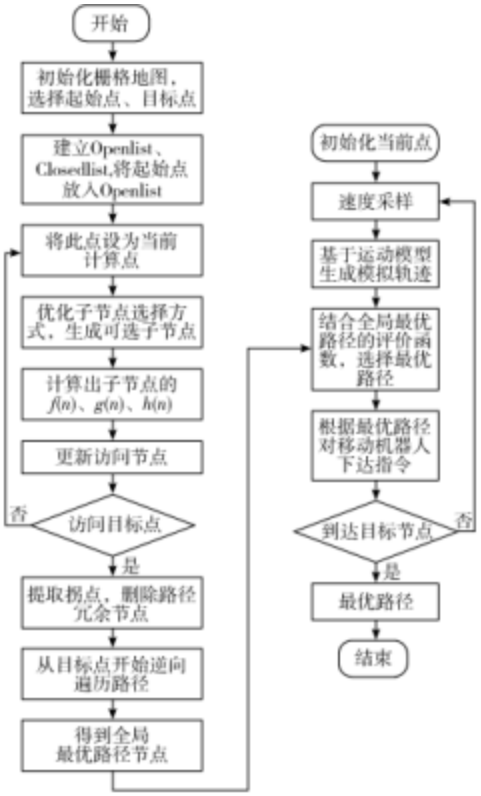


图 4　融合算法流程图

全局路径规划和dwa融合的算法流程

与A*优化类似，我们也需要对dwa的评价函数进行优化。现有的评价函数

$$G(v,w)=\sigma(\alpha heading(v,w)+\beta distance(v,w)+\gamma velocity(v,w))$$

其中 $heading(v,w)$ 为机器人方位角评价子函数　$distance(v,w)$ 为机器人与障碍物距离评价子函数　$velocity(v,w)$ 为机器人速度评价子函数

具体的含义也就不在这里列举了。

3.22teb

哈工大威海创新平台 - 718-夏天怡(22970116)

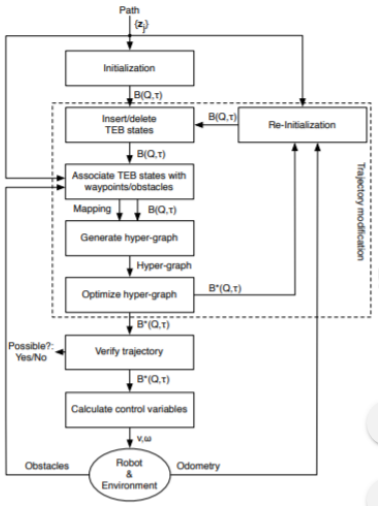


Figure 6: Control flow of TEB-implementation

from[5]

好消息是ROS已经提供了相关的局部路径规划包，可以直接下载使用

dwa_local_planner

teb_local_planner

3.3基于数学的路径优化

3.31多项式轨迹

$$p(t) = p_0 + p_1 t + p_2 t^2 \ldots + p_n t^n = \sum_{i=0}^{n} p_i t^i$$

$$p(t) = [1, t, t^2, \ldots, t^n] \cdot p$$

$$p = [p_0, p_1, \ldots, p_n]^T$$

$$v(t) = p'(t) = [0, 1, 2t, 3t^2, 4t^3, \ldots, nt^{n-1}] \cdot p$$

$$a(t) = p''(t) = [0, 0, 2, 6t, 12t^2, \ldots, n(n-1)t^{n-2}] \cdot p$$

$$jerk(t) = p^{(3)}(t) = [0, 0, 0, 6, 24t, \ldots, \frac{n!}{(n-3)!}t^{n-3}] \cdot p$$

$$snap(t) = p^{(4)}(t) = [0, 0, 0, 0, 24, \ldots, \frac{n!}{(n-4)!}t^{n-4}] \cdot p$$

- jerk(t),snap(t)

$$p(t) = \begin{cases} [1, t, t^2, \ldots, t^n] \cdot p_1 & t_0 \le t < t_1 \\ [1, t, t^2, \ldots, t^n] \cdot p_2 & t_1 \le t < t_2 \\ \ldots \\ [1, t, t^2, \ldots, t^n] \cdot p_k & t_{k-1} \le t < t_k \end{cases}$$

$$p_i = [p_{i_0}, p_{i_1}, \ldots, p_{i_n}]^T$$

$$p = [p_1^T, p_2^T, \ldots, p_k^T]^T$$

- 用k表示轨迹段数：
- 关键在于求解参数p，确定轨迹

- 基本要求：
  - 两端轨迹之间连续
  - 轨迹经过固定点：航路点
  - 轨迹无碰撞
- 高级要求：
  - 轨迹最顺滑、能量最优等

$$\begin{aligned} &\min f(p) \\ s.t.\ \ &A_{eq}p = b_{eq}, \\ &A_{ieq}p \le b_{ieq} \end{aligned}$$

- 等式约束和不等式约束

3.32MinimumSnap

> Vijay Kumar的一系列论文：Mellinger D, Kumar V. Minimum snap trajectory generation and control for quadrotors[C]//Robotics and Automation (ICRA), 2011 IEEE International Conference on. IEEE, 2011: 2520–2525.

优化目标：

$$\begin{aligned} &minimum\ snap: \ \min f(p) = \min(p^{(4)}(t))^2 \\ &minimum\ jerk: \ \min f(p) = \min(p^{(3)}(t))^2 \\ &minimum\ acce: \ \min f(p) = \min(p^{(2)}(t))^2 \end{aligned}$$

主要的思想是：加速度较小时，速度和变化较小，这样轨迹就不容易发生很大的变化。

3.4PRM(概率路线图)

PRM是一种基于图搜索的方法，它将连续空间转换成离散空间，再利用A*等搜索算法在路线图上寻找路径。这种方法能用相对少的随机采样点来找到一个解，对多数问题而言，相对少的样本足以覆盖大部分可行的空间，并且找到路径的概率为1（随着采样数增加，P（找到一条路径）指数的趋向于1）。显然，当采样点太少，或者分布不合理时，PRM算法是不完备的，但是随着采用点的增加，也可以达到完备。

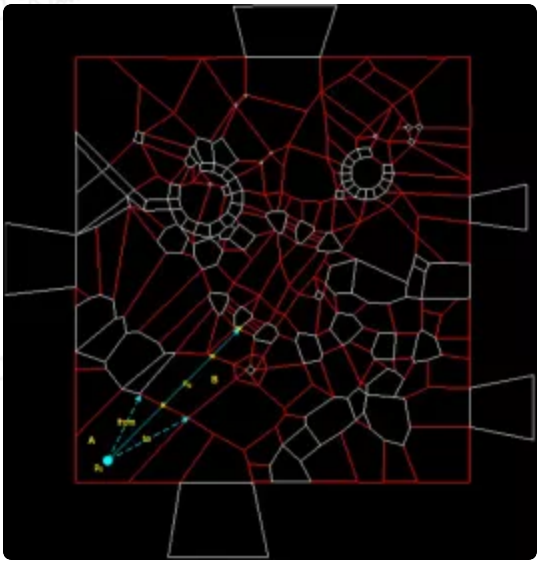
3.5常见路径规划算法的总结：

算法类型	具体算法	完备性	是否最优
Search_based	dijkstra,A*	完备	是
智能算法	遗传算法,蚁群算法	完备	否
Sample_based	PRM,RRT	概率完备	否
	RRT*,Informed RRT*	概率完备	渐进最优

在速度方面，
RRT系列>A*算法>dijkstra>智能算法

3.6Navigation Mesh

我们设一个数据结构(c, p, k, B, pid)来作为一个存储单元。c代表当前正在探测的凸多边形；p代表当前凸多边形的起始点*（可以理解为当前多边形的行走/导航节点）*；k代表过往路径的斜率*（原文即slope，可以理解为路径方向）*；B是一个布尔值，代表路径方向是否改变；pid代表当前节点在该路径上的前驱节点。

 这样的一个凸多边形地图

我们设导航的起始点为s，终点为t，起始点所在的凸多边形为cs，终点多边形为ct，初始路径斜率为k0，k0代表s到t点连线的斜率。
完整的算法流程如下：

1. 初始化一个节点(c_s, s, k0, TRUE, -1)，并创建一个空的openset，将这个节点加入Open集合中。 创建一个空的closeset。
2. 如果Open集合为空，则退出算法，并返回寻路失败信息。
3. 在Open集合中取出并移除第一个节点*(fmin)*，然后将这个节点放入closeset中。我们将这个节点表示为(c_n, p_n, k_n, B_n, pid_n)。

4. 如果c_n是终点凸多边形，则寻路成功。可以直接通过c_n的pid信息追踪路径，直至pid = –1，即回到起始点。
5. 如果c_n不是终点凸多边形，则检测c_n的所有相邻多边形。
6. 然后就是计算gScore，即从起始节点到当前凸多边形的确切距离：我们用斜率k_n来对p_n进行检测。如果存在可以通过路径斜率线连通的凸多边形，则将其设为c_n+1，并将斜率线与该凸多边形的交点设为p_n+1。计算出p_n到p_n+1的距离，加上p_n节点的G值，即可作为p_n+1的G值。最后，将B_n+1设为TRUE（注：即没有改变方向）。
7. 对于其余的相邻凸多边形，我们采用它们和当前凸多边形c_n的邻边中点作为p_n+1，仿效上面的方法即可计算出到p_n+1节点的G值。最后，将B_n+1设为FALSE。
8. 接下来让我们来计算hScore，我们会对改变路径方向的情况进行惩罚：

对于没有改变路径方向的凸多边形，我们简单采用p_n到终点t的直线距离作为H值。

对于那些改变了路径方向的凸多边形，我们采用上述H值乘以(1+w)（w为参数）。

9. 计算f=g+h，忽略掉那些已经在Closed集合中的凸多边形，将尚未在Open集合中的凸多边形加入Open集合中。对于那些已经在Open集合中的凸多边形，检测当前凸多边形c_n是否是一个更好的前驱，即G值更小。若是，则将c_n更新为它的前驱，并更新G值和F值。
10. 注意我们仍然可以利用二叉搜索树来作为openset

在这样的一个地图中，我们是这样来做line of sight detect的：

（白线包围的区域代表障碍，不可进入）

我们以初始位于p0点为例：

1. 确定p0位于哪一个多边形中（利用叉乘法）
2. 在该多边形中找射线r的穿出边和穿出点，并且得到穿出边所进入的另一个多边形
3. 直接对下一个多边形做障碍和边缘判断即可

这样的LOS检测是从初始多边形出发，沿着视线方向逐步遍历经过的多边形，这样在这个视线的引导下我们就可以很方便的省去一些空间上的数学处理。

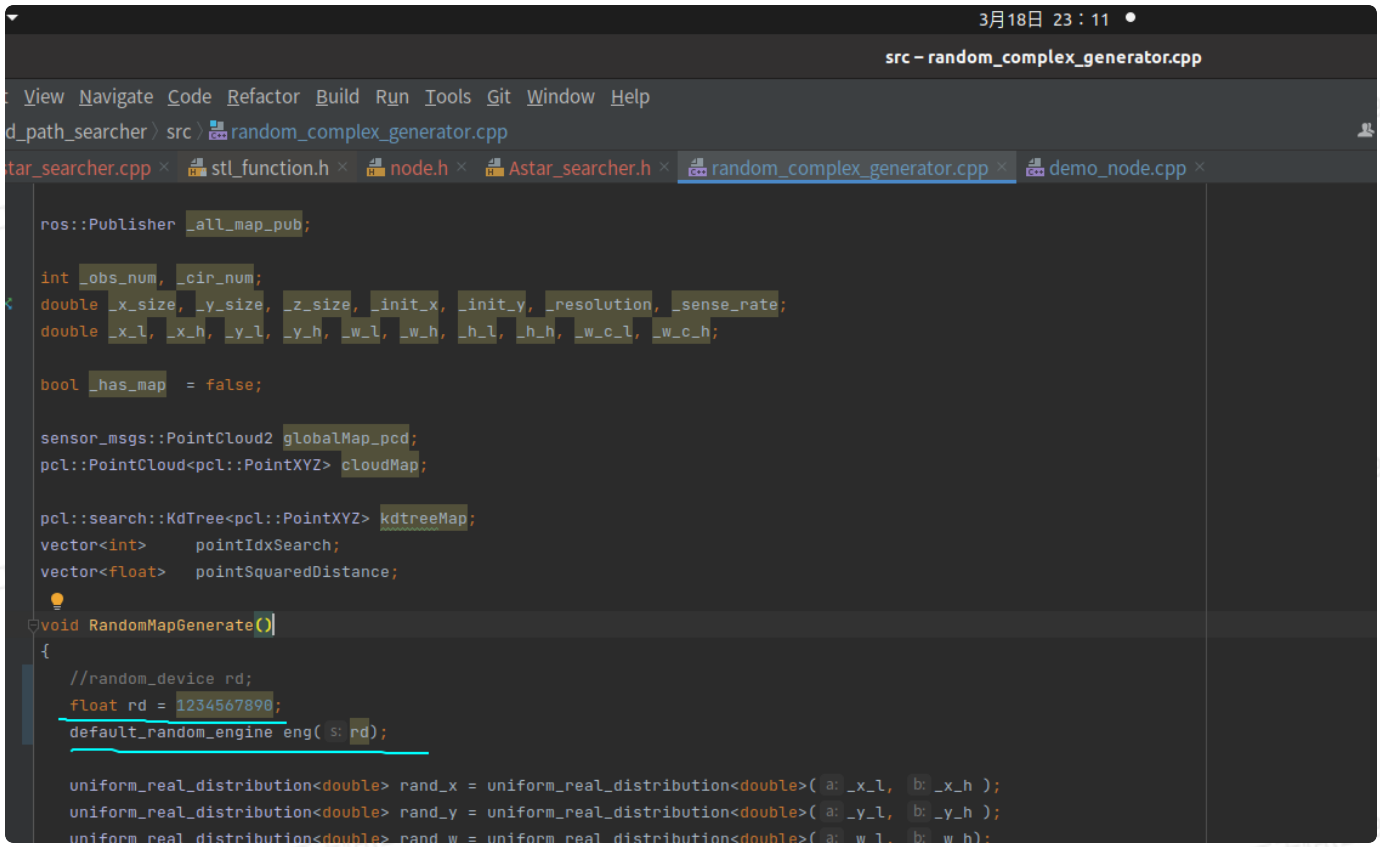
4.一些调试和比较的小技巧

4.1地图固定

由于在完成修改w或者改变启发函数等对原有算法改动不大的测试时，不需要同时对多条路径进行显示，于是我利用固定地图的方式简单地实现比较的操作。

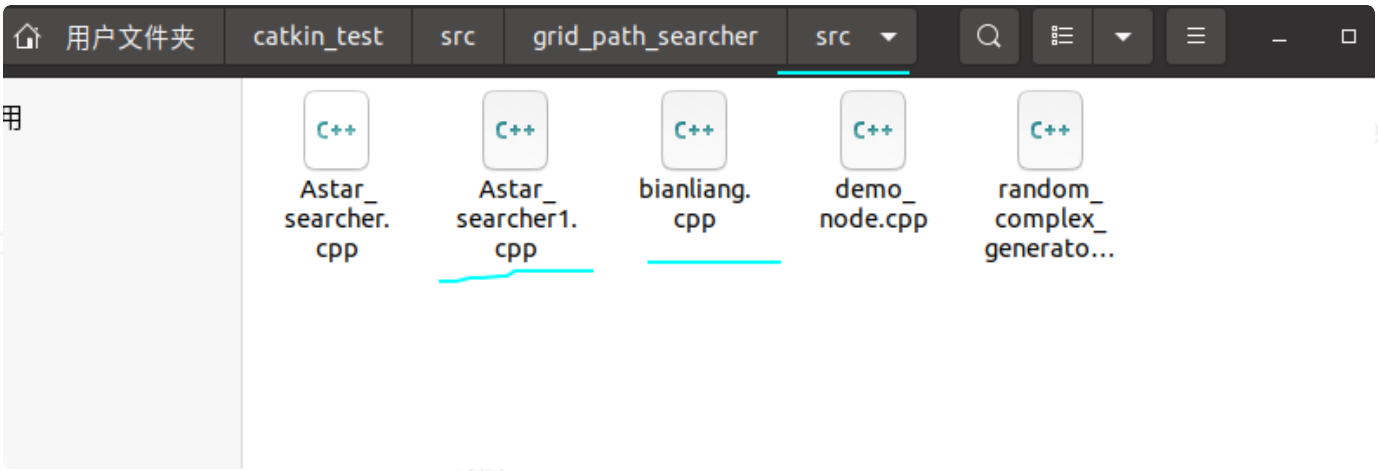
哈工大威海创新平台 - 718-夏天怡(22970116)

在 `random_complex_generator.cpp` 里将随机数种子固定就可以了。

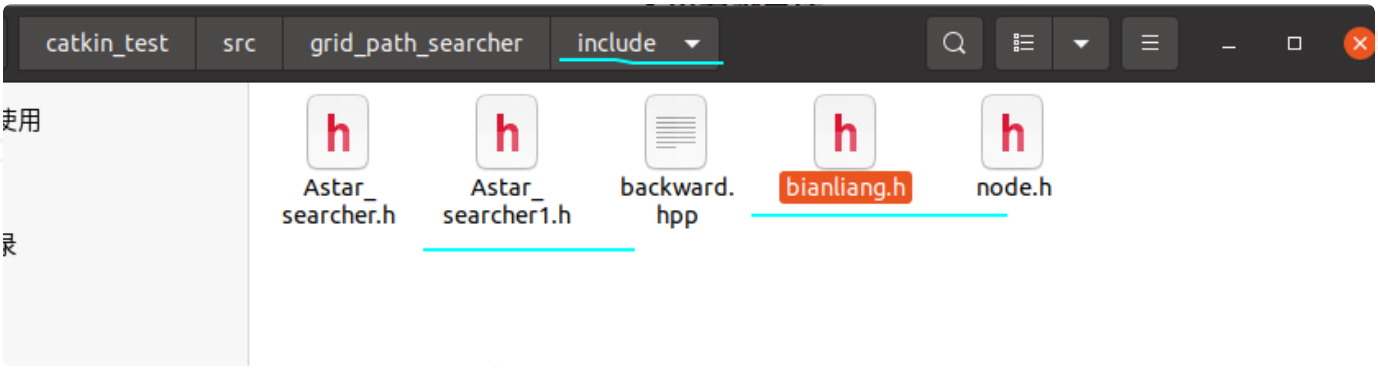


4.2同时显示路径

1.首先把优化代码的cpp文件放入src中



2.再把.h文件放入include中



这里要注意的是由于优化的代码更新了node.h文件（向结构体中添加了一些信息），所以这里node.h复制过来的时候直接替换原有文件即可。

3.接下来就是修改 `demo.cpp` 的文件

添加如下代码：

哈工大威海创新平台 - 718-夏天怡(22970116)

C++ 复制代码

```cpp
//注释部分为新添加的代码

AstarPathFinder * _astar_path_finder     = new AstarPathFinder();
//AstarPathFinder1 * _astar_path_finder1  = new AstarPathFinder1();

void rcvPointCloudCallBack(const sensor_msgs::PointCloud2 &
pointcloud_map)
{
    ......
    for (int idx = 0; idx < (int)cloud.points.size(); idx++)
    {
        pt = cloud.points[idx];

        _astar_path_finder->setObs(pt.x, pt.y, pt.z);
        //_astar_path_finder1->setObs(pt.x, pt.y, pt.z);
        //这里我一开始忘记添加，会产生在优化的路径不识别障碍物的情况，“穿墙而入”
        ......
    }
}
void pathFinding(const Vector3d start_pt, const Vector3d target_pt)
{
/*
#define _use_update 1
#if _use_update
    {
        _astar_path_finder1->AstarGraphSearch(start_pt, target_pt);

        auto grid_path     = _astar_path_finder1->getPath();
        auto visited_nodes = _astar_path_finder1->getVisitedNodes();

        visGridPath (grid_path,_use_update);

        visVisitedNode(visited_nodes);

        _astar_path_finder1->resetUsedGrids();
    }
#endif */

    _astar_path_finder->AstarGraphSearch(start_pt, target_pt);

    auto grid_path     = _astar_path_finder->getPath();
    auto visited_nodes = _astar_path_finder->getVisitedNodes();

    visGridPath (grid_path, false);

```

```
45        visVisitedNode(visited_nodes);
46
47        _astar_path_finder->resetUsedGrids();
48    }
49    int main(int argc, char** argv)
50    {
51        ......
52
53        _astar_path_finder  = new AstarPathFinder();
54        _astar_path_finder  -> initGridMap(_resolution, _map_lower,
      _map_upper, _max_x_id, _max_y_id, _max_z_id);
55
56        //_astar_path_finder1  = new AstarPathFinder1();
57        //_astar_path_finder1  -> initGridMap(_resolution, _map_lower,
      _map_upper, _max_x_id, _max_y_id, _max_z_id);
58
59        ......
60
61        delete _astar_path_finder;
62        //delete _astar_path_finder1;
63        return 0;
64    }
65    void visGridPath( vector<Vector3d> nodes, bool is_use_update)
66    {
67        ......
68
69        //if(is_use_update)
70            node_vis.ns = "demo_node/astar_path1";
71        else
72            node_vis.ns = "demo_node/astar_path";
73        ......
74        //if(is_use_update){
75            node_vis.color.a = 1.0;
76            node_vis.color.r = 1.0;
77            node_vis.color.g = 1.0;
78            node_vis.color.b = 1.0;
79        }
80        else{
81            node_vis.color.a = 1.0;
82            node_vis.color.r = 0.0;
83            node_vis.color.g = 1.0;
84            node_vis.color.b = 0.0;
85        }
86        ......
87    }
88
```

4.在优化的cpp代码中把类名修改成新的。

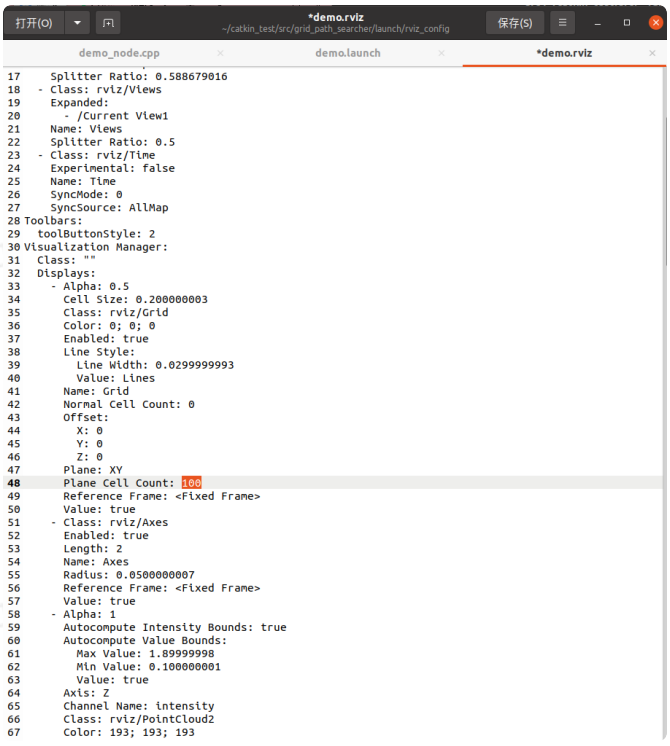
5.最后别忘了在CMakelists中

```
add_executable( demo_node
    src/demo_node.cpp
    src/Astar_searcher.cpp
    src/Astar_searcher1.cpp
    src/bianliang.cpp
    )
```

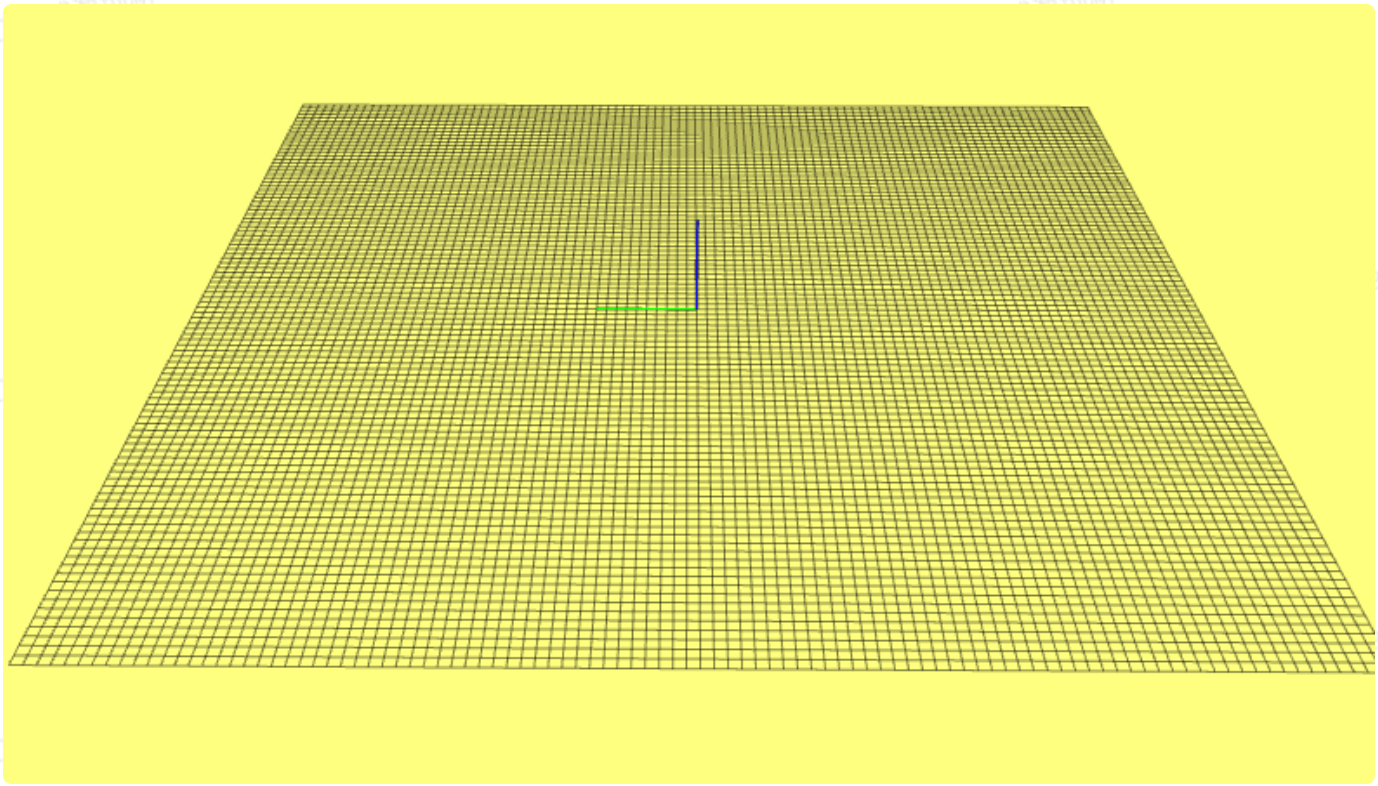
4.3调整地图大小和起点

因为在原来的设置下，路径太短，优化效果不明显，于是我修改了地图大小和起点位置

地图大小：



这个是rviz初始显示地图大小的更改，修改效果如下：



然后修改：

打开(O) *demo.launch ~/catkin_test/src/grid_path_searcher/launch 保存(S)

demo_node.cpp | *demo.launch | demo.rviz

```
1 <launch>
2
3 <arg name="map_size_x" default="20.0"/>
4 <arg name="map_size_y" default="20.0"/>
5 <arg name="map_size_z" default=" 2.0"/>
6
7 <arg name="start_x" default=" 0.0"/>
8 <arg name="start_y" default=" 0.0"/>
9 <arg name="start_z" default=" 1.0"/>
10
11   <node pkg="grid_path_searcher" type="demo_node" name="demo_node" output="screen" required =
  "true">
12       <remap from="~waypoints"       to="/waypoint_generator/waypoints"/>
13       <remap from="~map"             to="/random_complex/global_map"/>
14
15       <param name="map/margin"       value="0.0" />
16       <param name="map/resolution"   value="0.2" />
17       <param name="map/x_size"       value="$(arg map_size_x)"/>
18       <param name="map/y_size"       value="$(arg map_size_y)"/>
19       <param name="map/z_size"       value="$(arg map_size_z)"/>
20
```

哈工大威海创新平台 - 718-夏天怡(22970116)

起点位置：

打开(O) *demo.launch ~/catkin_test/src/grid_path_searcher/launch 保存(S)

CMakeLists.txt | demo_node.cpp | *demo.launch

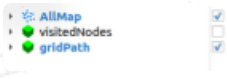
```
<launch>

<arg name="map_size_x" default="20.0"/>
<arg name="map_size_y" default="20.0"/>
<arg name="map_size_z" default=" 2.0"/>

<arg name="start_x" default=" 10.0"/>
<arg name="start_y" default=" 10.0"/>
<arg name="start_z" default=" 1.0"/>

  <node pkg="grid_path_searcher" type="demo_node" name="demo_node" output="screen" required =
"true">
      <remap from="~waypoints"       to="/waypoint_generator/waypoints"/>
      <remap from="~map"             to="/random_complex/global_map"/>
```

4.4 visited_nodes

```
vector<Vector3d> AstarPathFinder::getVisitedNodes()
{
    vector<Vector3d> visited_nodes;
    for(int i = 0; i < GLX_SIZE; i++)
        for(int j = 0; j < GLY_SIZE; j++)
            for(int k = 0; k < GLZ_SIZE; k++){
                if(GridNodeMap[i][j][k]->id == -1)
                    visited_nodes.push_back(GridNodeMap[i][j][k]->coord);
            }

    ROS_WARN("visited_nodes size : %d", visited_nodes.size());
    return visited_nodes;
}
```

官方给的visited_nodes的含义是进入closeset的节点，但是我认为这是有问题的，而将条件改成id!=-1才是有意义的，因为这样就表示我曾经搜索过并添加到openSet里的点的个数（含被放入closeset）

4.5rviz显示visited nodes



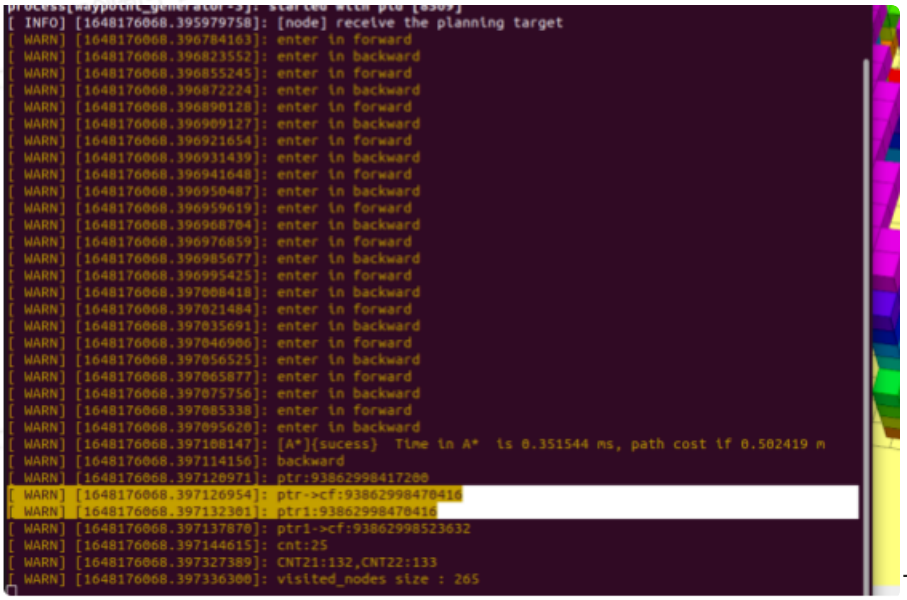
勾选对应的小方框就可以

4.6利用ROS_WARN来调试

在一开始优化A*的过程中，代码很容易出错，而且最终运行的结果无外乎是没有找到路径的：

> Segmentation fault (Address not mapped to object [0x48])
> ==========
> ==========REQUIRED process [demo_node-1] has died!

等等显示，只看这个结果是没有办法轻易地找到自己代码出错的位置的。于是在我调试优化的时候，通过在关键位置处设置ROS_WARN指令来在命令行窗口打印一些必要的变量值和提示信息来判断程序执行的情况。
以下是我在完成双向A*优化时的一些打印信息：



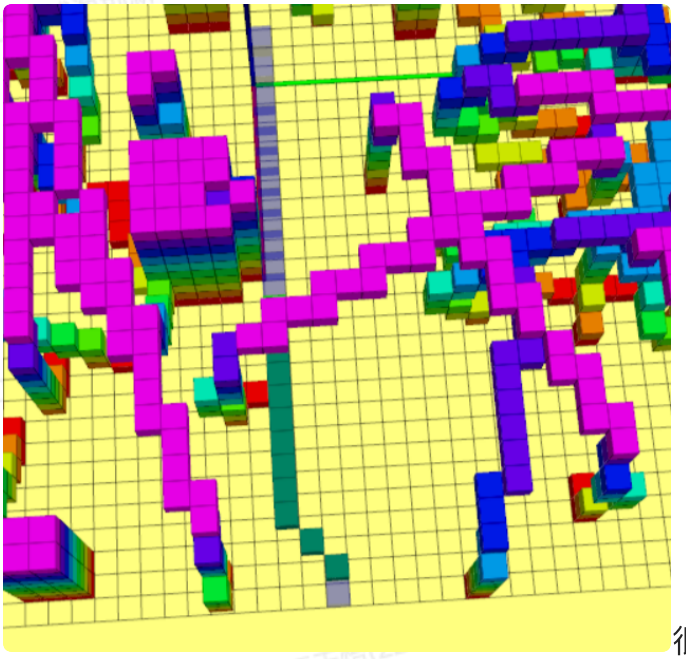
可以看到正向与返向的搜索是交替进行的。并且我也通过这个方式找到了我双向A*之前运行失败的原因，见5.1

5.错误修正

5.1双向A*

实现这个算法的过程中确实遇到了很多问题，好在我将它们一一解决。

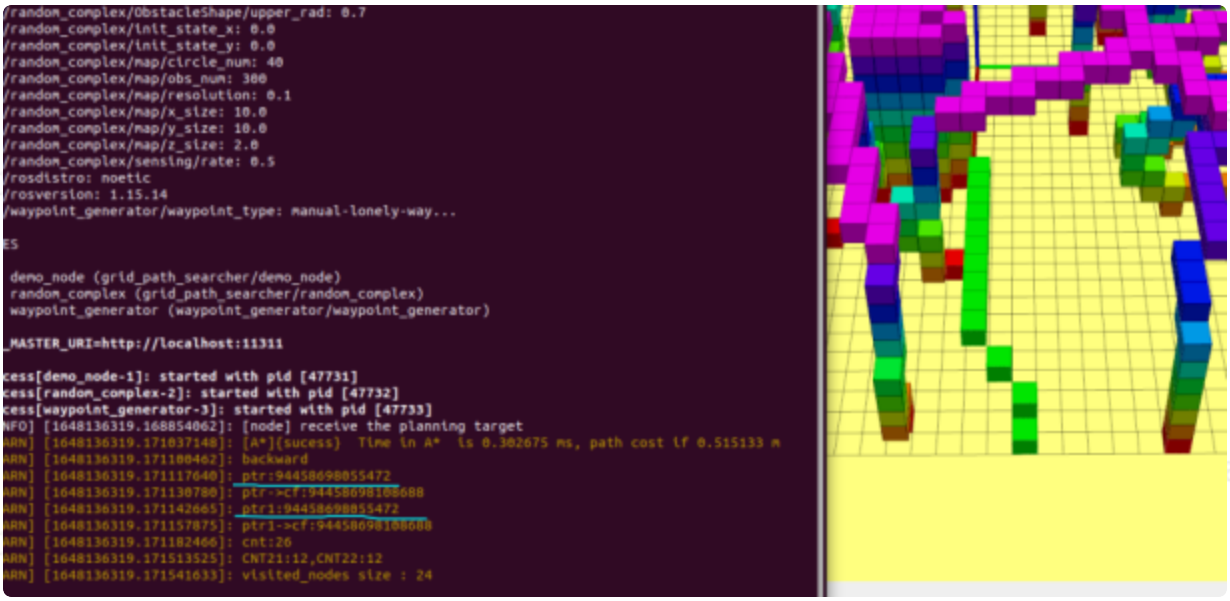
1. 对查找列表的数据结构的选取：
    a. 最开始考虑使用 `unordered_map` 去存，它内部是一个hashtable，也是用key去查找value，但是问题在于我们的查找表只需要存节点的ptr即可，所以用它存也是不方便的，并且该数据结构内部是无序的，这样
    b. 最后我就选择了简单的 `vector` 去存储
2. 在成功编译之后，在路径显示中却出现了问题：



很明显它只显示了某一方向的路径，我一开始以为是路径搜索方面出现了问题，后来细想并不会，因为这样的话，命令行窗口就会报错，因为没有找到路径，而这样既然可以显示那就说明路径是找到了，但是显示不对。

a. 首先考虑vis_，发现没什么问题。

b. 然后通过添加调试信息来跟踪程序的运行情况。

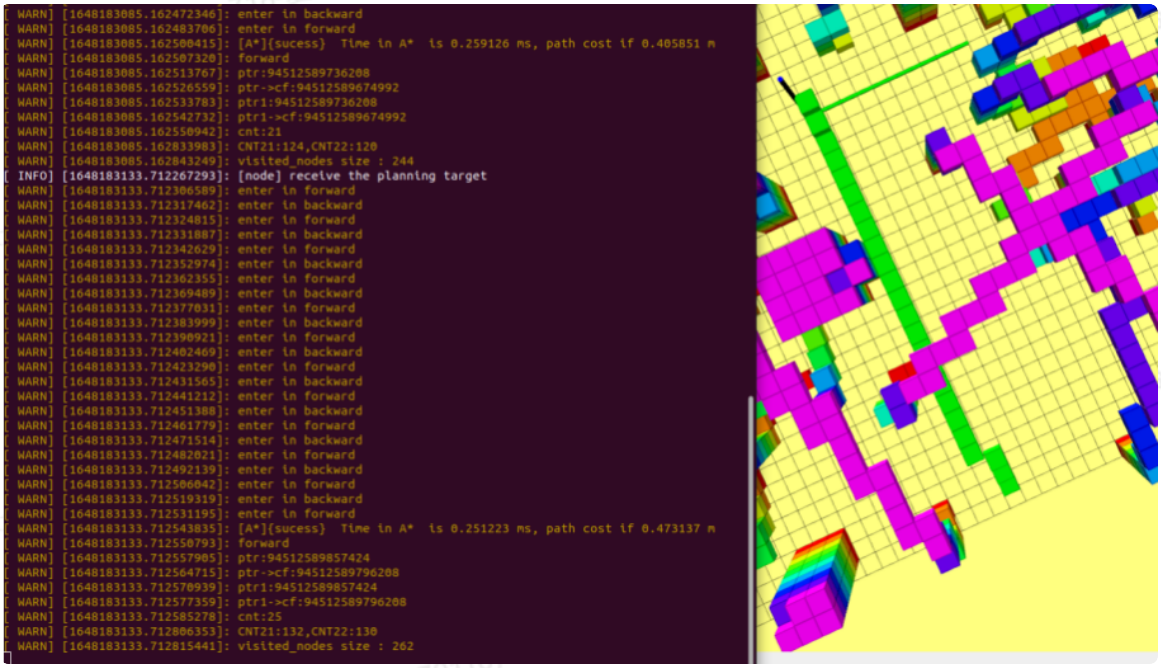


可以看到最后的terminatePtr和terminatePtr1指向了同一个东西，那根据它们两向上回溯父节点不就当然是一个方向吗！于是我仔细地思考了自己停止搜索的终止条件：

> 如果在正向查找表fopenSet中找到了此时正向搜索的目标点(recrtPtr)，或者在反向查找表freopenSet中找到了此时反向搜索的目标点(crtPtr)，查找终止。以正向搜索时停止为例，将recrtPtr在fopenSet里对应的节点p找到以后（这两个节点实际上是同一个节点但是它们的

> camefrom不一样），返回该节点p和recrtPtr给getPath，由p回溯父节点，得到正向路径，由recrtPtr回溯父节点，得到反向路径。（引用自2.11）

我的本意是给停止搜索时的正向节点和反向节点之间建立联系，但是如果按我最初的代码来看，这样的效果是让最后开始回溯的两个节点为同一个！于是，我查看之前的代码，发现，对于任意节点，它的父节点属性只有唯一一个 `camefrom` ，那么这样就会造成（以正向搜索为例）如果令terminatePtr = p(recrtPtr在fopenSet里的对应点），terminatePtr1 = recrtPtr的话，他们不就是同一个点吗！这样回溯唯一的camefrom那必然一样。我考虑再三，在不改变终止条件的情况下，实际上只需要为节点添加另一个 `camefrom1` 属性就可以了，而它是在反向搜索中更新的，那么在最后回溯父节点的时候，反向的节点回溯的是camefrom1，正向系统的节点回溯的是camefrom这样的话路径就完全分为两个方向了，好耶。



以上是成功后的显示信息和路径，可以看到ptr虽然等于ptr1，但是他们的camefrom确实是不一样的。

5.2编译的一些问题

1. 首先是catkin_make一堆pcl::,std::的问题，我一开始以为是pcl库没有安装的问题，但是经检查后，确实有pcl库，由于出现了std的问题猜想是c++的版本兼容问题，于是在src里的每个CMakelists中添加

哈工大威海创新平台 - 718-夏天怡(22970116)

打开(O)

CMakeLists.txt
~/catkin_ws/src/waypoint_generator

```
cmake_minimum_required(VERSION 2.8.3)
project(waypoint_generator)
set(CMAKE_CXX_STANDARD 11)
set(CMAKE_VERBOSE_MAKEFILE "true")
include(CheckCXXCompilerFlag)
CHECK_CXX_COMPILER_FLAG("-std=c++11" COMPILER_SUPPORTS_CXX11)
CHECK_CXX_COMPILER_FLAG("-std=c++0x" COMPILER_SUPPORTS_CXX0X)
if(COMPILER_SUPPORTS_CXX11)
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11")
elseif(COMPILER_SUPPORTS_CXX0X)
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++0x")
else()
        message(STATUS "The compiler ${CMAKE_CXX_COMPILER} has no C++11 support. Please use a different C++ compiler.")
endif()

set(ADDITIONAL_CXX_FLAG "-Wall -O3 -march=native")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} ${ADDITIONAL_CXX_FLAG}")

find_package(catkin REQUIRED COMPONENTS
  roscpp
  tf
  nav_msgs
)
catkin_package()

include_directories(include)
include_directories(
  ${catkin_INCLUDE_DIRS}
)

add_executable(waypoint_generator src/waypoint_generator.cpp)

target_link_libraries(waypoint_generator
   ${catkin_LIBRARIES}
)
```

然后这个问题大概就解决了。

2. 环境变量最好是直接放在.bashrc中这样可以保证不会出错。
3. 打开rviz发现没有地图出现并且有allmaperror的情况，Failed to transform from frame [/world] to frame [world]在群里询问，于是在demo_node.cpp中将/world改为world后正常。
4. 编译过程中如果出现问题建议重新配置工作环境，要不然再编译的时候很可能还有之前的错误（然后也不知道错哪）

5.3模拟退火

之前给出的模拟退火使用方式其实是有一些大材小用的，因为它实际上也是在某一次搜索中给予一定的打破A*基于fscore的最小值拓展，并且引导搜索方向的一个小优化，实际上模拟退火的方法可以从宏观上（整条路径出发）来做出对路径的优化。例如，某次A*得到的路径作为迭代的初始解，然后随机置换其中的几个节点得到新解，再利用Metropolis准则判断新解是否可以被接受，然后重复该过程直至T降低到一定程度后停止迭代。不过这也显然是一种牺牲时间的方法。

5.4getAngle

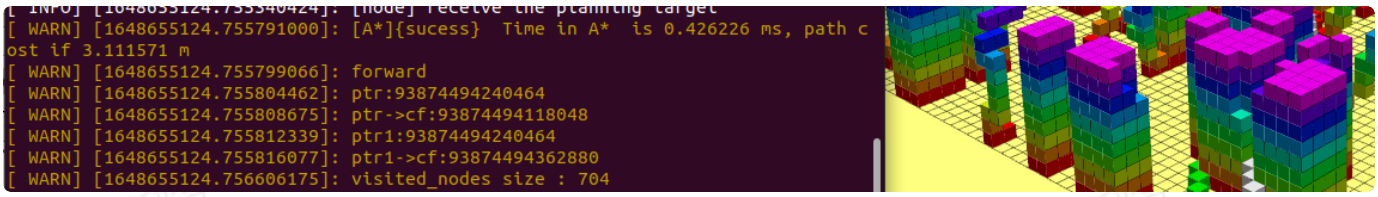
一开始getAngle是利用反三角函数计算两点之间连线与水平面的夹角的，但是这样计算出来的角度是没有意义的，因为我们实际上要求的是机器人的转向角，也就是两条有向线段的夹角，所以我利用向量的点乘来求这个夹角。

三、算法部署

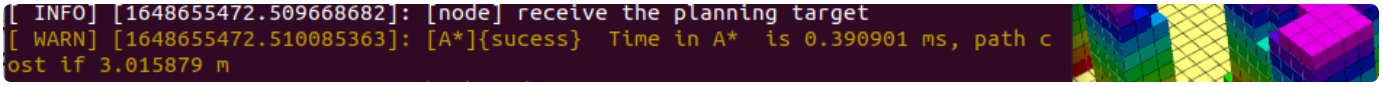
主体使用双向A*时，发现他的耗时挺长的，有点不理解，然后优化了一下查找表的处理，原来：是从fopenset的首个元素开始查找的，这样其实浪费了很多时间，因为很显然我们要查找的节点大概率是在之后才被添加进列表的，所以我使用反向迭代器，从list末尾开始查找：

```
GridNodePtr AstarPathFinder1::isinvector(std::list<GridNodePtr> openset, GridNodePtr ptr)
{
    //std::list<GridNodePtr>::iterator itr=openset.begin();
    //for(;itr!=openset.end();itr++)
    std::list<GridNodePtr>::reverse_iterator itr = openset.rbegin();
    for(;itr!=openset.rend();itr++)
    {
        if((*itr)->index(0) == ptr->index(0)&&(*itr)->index(1) == ptr->index(1)&&(*itr)->index(2) =
            //auto p1 = *itr;
            p = (*itr);
            flagp=1;
            break;
        }
    }
    //ROS_WARN("[A*]{failed}  p  is %f ",(*itr));
    //return p;
    return p;
}
```

优化前：

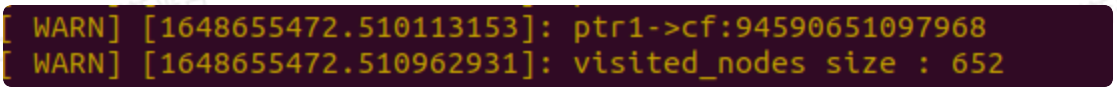


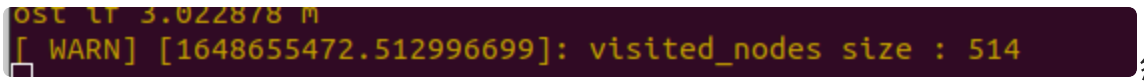
优化后



不过也没有优化太多。。

还有一个我不能接受的是，双向A*的visited_nodes比原来多。。

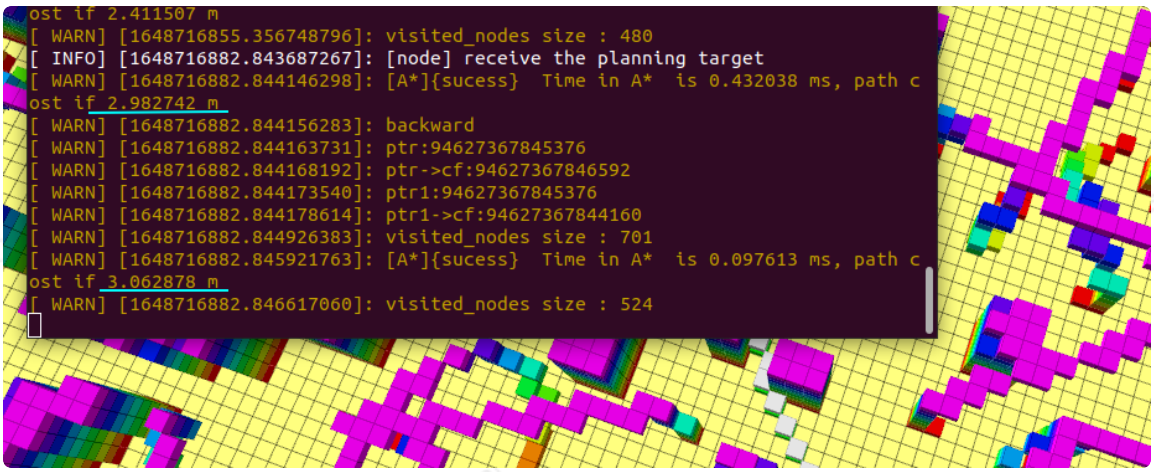




然后我检查了我的getVisitedNodes的函数：

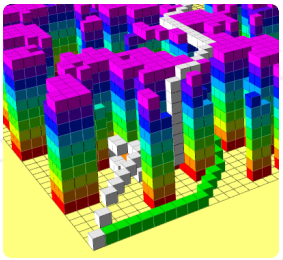
```cpp
vector<Vector3d> AstarPathFinder1::getVisitedNodes()
{
    int cnt21=0;
    int cnt22=0;
    vector<Vector3d> visited_nodes;
    for(int i = 0; i < GLX_SIZE; i++)
        for(int j = 0; j < GLY_SIZE; j++)
            for(int k = 0; k < GLZ_SIZE; k++){
                //if(GridNodeMap[i][j][k]->id == -1||GridNodeMap[i][j][k]->rid == -1)
                //if(GridNodeMap[i][j][k]->id == -1) {
                if(GridNodeMap[i][j][k]->id !=0) {
                    //if(GridNodeMap[i][j][k]->rid == -1)
                    visited_nodes.push_back(GridNodeMap[i][j][k]->coord);
                    cnt21++;
                }
                else if(GridNodeMap[i][j][k]->rid !=0) {
                    visited_nodes.push_back(GridNodeMap[i][j][k]->coord);
                    cnt22++;
                }
                /*if(GridNodeMap[i][j][k]->rid !=0) {
                    visited_nodes.push_back(GridNodeMap[i][j][k]->coord);
                    cnt22++;
                }*/

            }
    //ROS_WARN("CNT21:%d,CNT22:%d",cnt21,cnt22);
    ROS_WARN("visited_nodes size : %d", visited_nodes.size());
    return visited_nodes;
}
```

这样可以避免对一个节点重复输入。



还遇到了一个这样的问题：

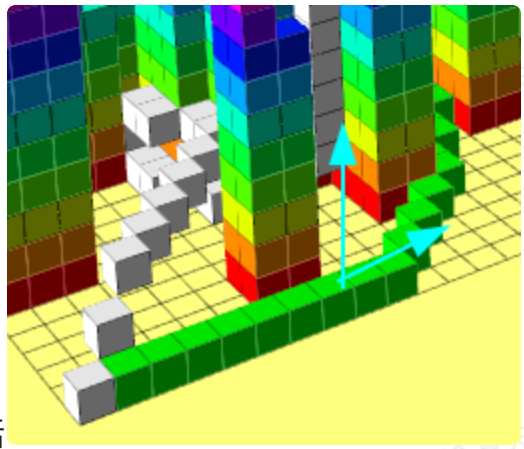
哈工大威海创新平台 - 718-夏天怡(22970116)



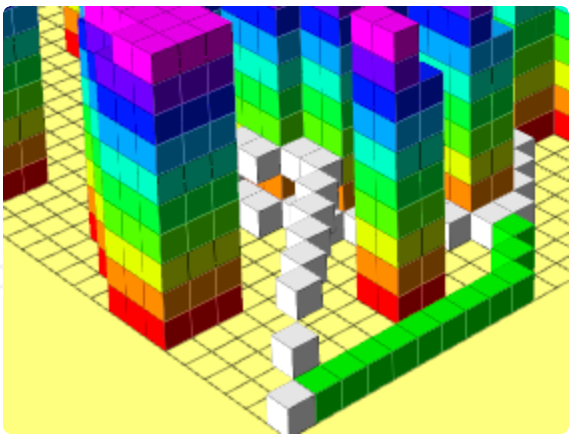
为什么呢：因为在这个时候我的启发函数是这样的：

```
if(dx>dy)
{
    h = 10*dx+5*dy;
}
else
    h = 5*dx+10*dy;
h+=7*dz;
```

这样会导致在出发的时候，水平面运动和上升的地位相比，上升的启发函数占比较低，所以路径会倾向于上升，这样的话



优化后的路径会更倾向于向上延伸，这样就会进入围墙中，导致被障碍物环绕，而原有的路径由于沿水平方向行走，就绕开了障碍，不会进入围墙中。所以我修改dz前面的系数：为15



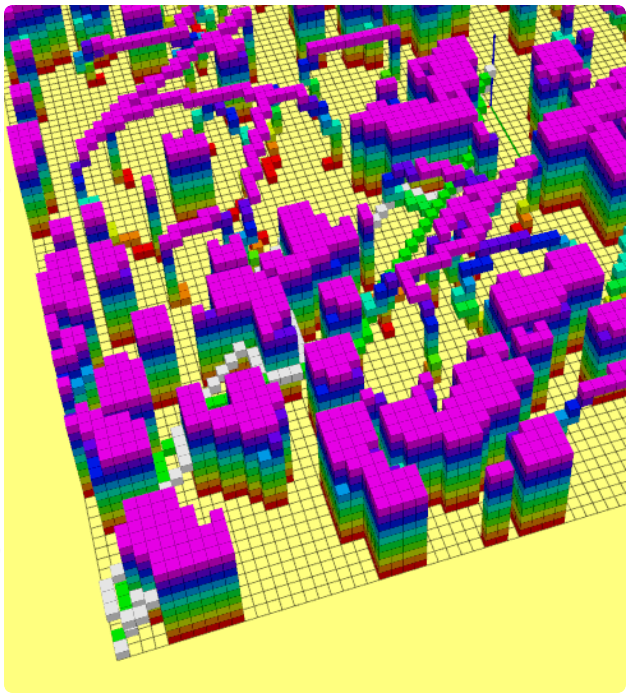
这样的话明显可以看出它不会沿dz一直上升了，但是还会有偏移的问题，为什么呢？其实也很好理解，因为优化后的Manhattan会倾向于沿着对角线移动（因为这个在多数情况下是最短的）

哈工大威海创新平台 - 718-夏天怡(22970116)

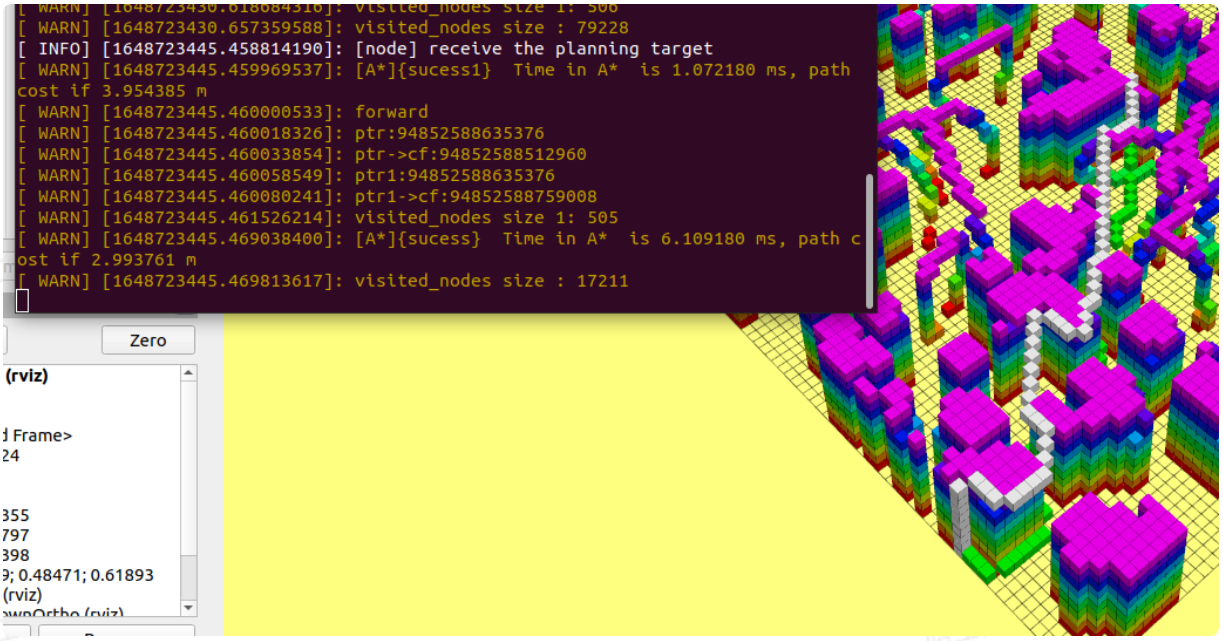
在调试的过程中，我对输出指令加了一点小优化：

```
[ INFO] [1648722763.025905540]: [node] receive the planning target
[ WARN] [1648722763.027077097]: [A*]{sucess1}  Time in A*  is 0.527403 ms, path
cost if 3.958434 m
[ WARN] [1648722763.027091284]: backward
[ WARN] [1648722763.027097763]: ptr:94852588508752
[ WARN] [1648722763.027102284]: ptr->cf:94852588507536
[ WARN] [1648722763.027107029]: ptr1:94852588508752
[ WARN] [1648722763.027111297]: ptr1->cf:94852588509968
[ WARN] [1648722763.027934706]: visited_nodes size 1: 807
[ WARN] [1648722763.032678964]: [A*]{sucess}  Time in A*  is 3.969474 ms, path c
ost if 3.140035 m
[ WARN] [1648722763.033422544]: visited_nodes size : 12105
```

注：此处遍历节点数增多是因为调大了地图尺寸和障碍物数量。

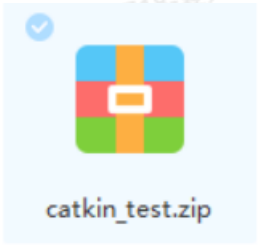


我们可以看到，在这样的障碍物很多的情况下，优化的算法取得了优异的成绩。



四、效果展示

1. 双向A*与启发函数优化：

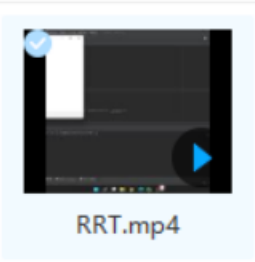


代码：

视频：

2. RRT实现：

代码：



视频：

视频和源码合集：链接：https://pan.baidu.com/s/1D-JTI1I3JNj1GBoXcw3gBw
提取码：xywi

五、未来优化与应用

其实pathplanning和motionplanning并不仅仅作用于机器人，在机械臂方面也有很多的应用，并且它们有很大的不同，机械臂更多的是一个多自由度的系统，需要考虑多个刚体的协同运动。
对于path planning，其实还有很多现代的规划方法，比如人工势场等，所以基于图搜索的算法只是冰山一角而已。
线下赛中，对于路径规划部分，更多的是使用官方给我们的ROS包，因此，在掌握如上算法以后，我还需要对ROS包的具体内容进行深入学习，并且与负责建图的同学合作，基于ROS实现小车的实时定位与建图，实现自主导航，以及避障。

六、总结

通过这次的线上赛学习，我最终优化的算法达到了：

1. 在环境复杂的情况下，在不以路径长度为代价的前提下，大大减少了搜索的时间
2. 遍历的节点数大大减少，有效地节省了存储空间
3. 在三维尺度上能够很好的平衡xyz的路径选择
4. 添加了转弯代价，在实际应用时减少小车的转弯次数
5. 数据结构使用上的优化

此外，学习并且通过其他方式实现的算法：

1. RRT系列
2. 在matlab上实现路径平滑

其余学习部分省略。
在此基础上，我阅读了国内外许多pathplanning的论文，对算法的发展历史与趋势有了一定的认识，并学习了有关课程，发现自己掌握的A*只不过是一个古老的算法，自己才刚算入了pathplanning和motionplanning的门吧！不过尽管线上赛告一段落，我还是会继续在这个领域继续学习的。

七、参考资料

[1]李宁. 面向家庭环境的移动机器人局部路径规划算法研究[D].哈尔滨工业大学,2018.
[2]Likhachev, Maxim et al. “ARA*: Anytime A* with Provable Bounds on Sub-Optimality.” NIPS(2003).
[3]Nash, Alex et al. “Lazy Theta*: Any-Angle Path Planning and Path Length Analysis in 3D.” SOCS(2010).
[4]Xiao, A., Tong, W.B., Yang, L., Zeng, J., Li, Z., & Sreenath, K. (2021). Robotic Guide Dog: Leading a Human with Leash-Guided Hybrid Physical Interaction. 2021 IEEE International Conference on Robotics and Automation (ICRA), 11470-11476.

[5]C. Roesmann, W. Feiten, T. Woesch, F. Hoffmann and T. Bertram, "Trajectory modification considering dynamic constraints of autonomous robots," ROBOTIK 2012; 7th German Conference on Robotics, 2012, pp. 1–6.
[6]Sven Koenig, Maxim Likhachev, David Furcy,Lifelong Planning A*,Artificial Intelligence,Volume 155, Issues 1—2,2004,Pages 93–146,ISSN 0004–3702,
[7] Aine, S., Swaminathan, S., Narayanan, V., Hwang, V., & Likhachev, M. (2016). Multi–Heuristic A*. The International Journal of Robotics Research, 35(1—3), 224—243.
[8]Stentz, A. (1997). Optimal and efficient path planning for partially known environments. In Intelligent unmanned ground vehicles (pp. 203–220). Springer, Boston, MA.
[9]王维, 裴东, & 冯璋. (2018). 改进 A* 算法的移动机器人最短路径规划. 计算机应用, 38(5), 1523–1526.
[10]祁玄玄, 黄家骏, & 曹建安. (2020). 基于改进 A* 算法的无人车路径规划. 计算机应用, 40(7), 2021.

哈工大威海创新平台 - 718-夏天怡(22970116)